DOCUMENTO DE ENTREGA

ServiNow

Aplicación móvil de solicitudes y ofertas de servicios

SEGUNDA FASE

Implementación funcional y evidencias

Santiago Dorado
Juan David Saza
Fabián López

Kotlin · Jetpack Compose · MVVM · Hilt
Datos de evaluación en memoria

Repositorio de código fuente

https://github.com/JDFaFa/ServiNow

Capturas obtenidas de la aplicación ejecutada en un emulador Android. No se utilizaron imágenes del prototipo como prueba de implementación.

01 · ALCANCE

Qué incluye esta entrega

Aplicación funcional

Solicitudes, ofertas, comentarios, calificaciones, perfiles, currículum, avisos, estadísticas, reputación, logros y moderación se conectan mediante un repositorio compartido en memoria. Los cambios de un usuario son visibles al cambiar a otro usuario de prueba dentro del mismo proceso.

Adaptaciones autorizadas por la consigna

No se exige base de datos, sesiones persistentes, gestión de permisos, mapas ni carga externa de imágenes. Por ello se usan cuentas y datos de prueba en memoria, imágenes ilustrativas incluidas con la app y ciudades de referencia. El mapa de búsqueda existente se conserva como complemento opcional.

Simulaciones identificadas

La recuperación utiliza un enlace de prueba dentro de la app: modifica la clave en memoria y no envía correos. El botón Google informa que requiere un proveedor externo; no se presenta como OAuth operativo. Las cuentas de moderador están precargadas en el repositorio, no en una base de datos.

Alcance de la evidencia

Cada ficha vincula un requisito con una captura real y describe la acción, resultado y adaptación respecto al prototipo. Los datos de personas y trabajos que aparecen son ficticios. Los requisitos técnicos exentos se identifican expresamente en el anexo.

Versión de código

Código evaluado: commit local b50159b, preparado para la rama main. Publicación remota pendiente: GitHub rechazó la escritura por falta de permisos de la integración. Antes de entregar, publicar los commits preparados y comprobar que este código y el PDF aparecen en el repositorio enlazado.

02 · EVALUACIÓN

Cómo probar la aplicación

Cuentas de ejemplo

Solicitante: valentina@gmail.com / ServiNow1!
Trabajador: andres@gmail.com / ServiNow1!
Otro usuario: laura@hotmail.com / ServiNow1!
Moderador: moderador@gmail.com / Moderador1!
Son claves públicas de demostración, sin relación con cuentas reales.

Recorrido principal

1. Entrar como Valentina y crear una solicitud.
2. Cambiar al moderador y verificarla o rechazarla con motivo.
3. Entrar como Andrés, ofertar, comentar y marcar interés.
4. Volver a Valentina: leer avisos, aceptar una oferta, finalizar y reseñar.
5. Consultar el perfil del trabajador, estadísticas y logros.

Registro y recuperación

Registrar un correo nuevo permitido, iniciar sesión con él y probar una clave incorrecta. Solicitar la recuperación, abrir el enlace simulado y establecer una clave nueva. El cambio se aplica al usuario en memoria.

Reinicio de datos

Cerrar el proceso y abrir la app restaura las cuentas y trabajos iniciales. Cambiar de pantalla o de usuario dentro del mismo proceso conserva los cambios. Las capturas pueden reflejar datos modificados por la secuencia de pruebas.

Integrantes

Santiago Dorado, Juan David Saza y Fabián López. No se atribuye revisión manual o contribuciones de Git a un integrante sin evidencia de que las haya realizado.

03 · EVIDENCIAS FUNCIONALES

RF-01 · Registro de usuarios



IMPLEMENTACIÓN

Se ingresan nombre, correo y contraseña con confirmación. Se valida el dominio permitido y la complejidad de la clave; no se admiten correos duplicados.

RESPECTO AL PROTOTIPO

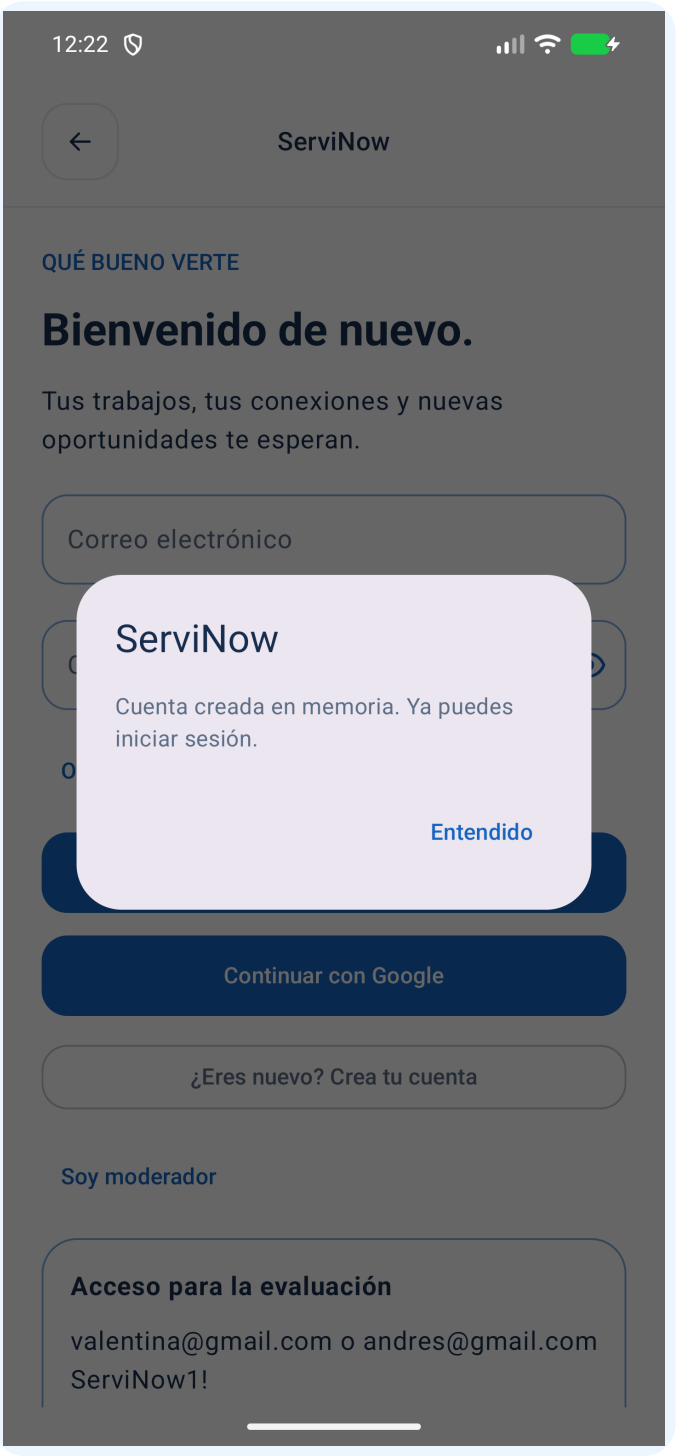
Una sola cuenta puede solicitar y ofrecer servicios. La creación ocurre en memoria; la dirección se completa después.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 02-register.png

03 · EVIDENCIAS FUNCIONALES

RF-02 · Confirmación de registro



IMPLEMENTACIÓN

El formulario ejecutado crea el usuario y muestra confirmación. La prueba continúa iniciando sesión con esa cuenta.

RESPECTO AL PROTOTIPO

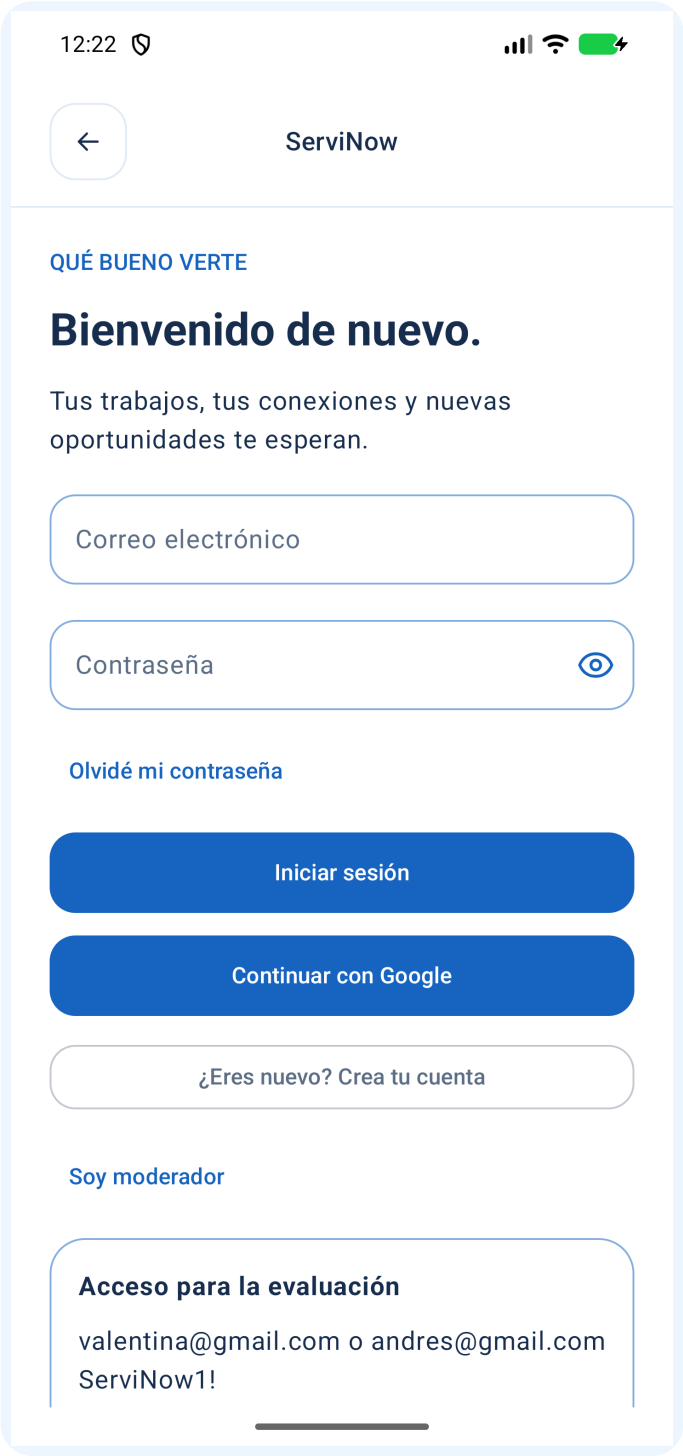
La confirmación reemplaza el aviso anterior que solo indicaba que faltaba conectar un servicio.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 03-registration-success.png

03 · EVIDENCIAS FUNCIONALES

RF-03 · Inicio de sesión y validación



IMPLEMENTACIÓN

El correo y la clave se comparan con las cuentas en memoria. Las credenciales incorrectas producen un diálogo; el rol determina la pantalla inicial.

RESPECTO AL PROTOTIPO

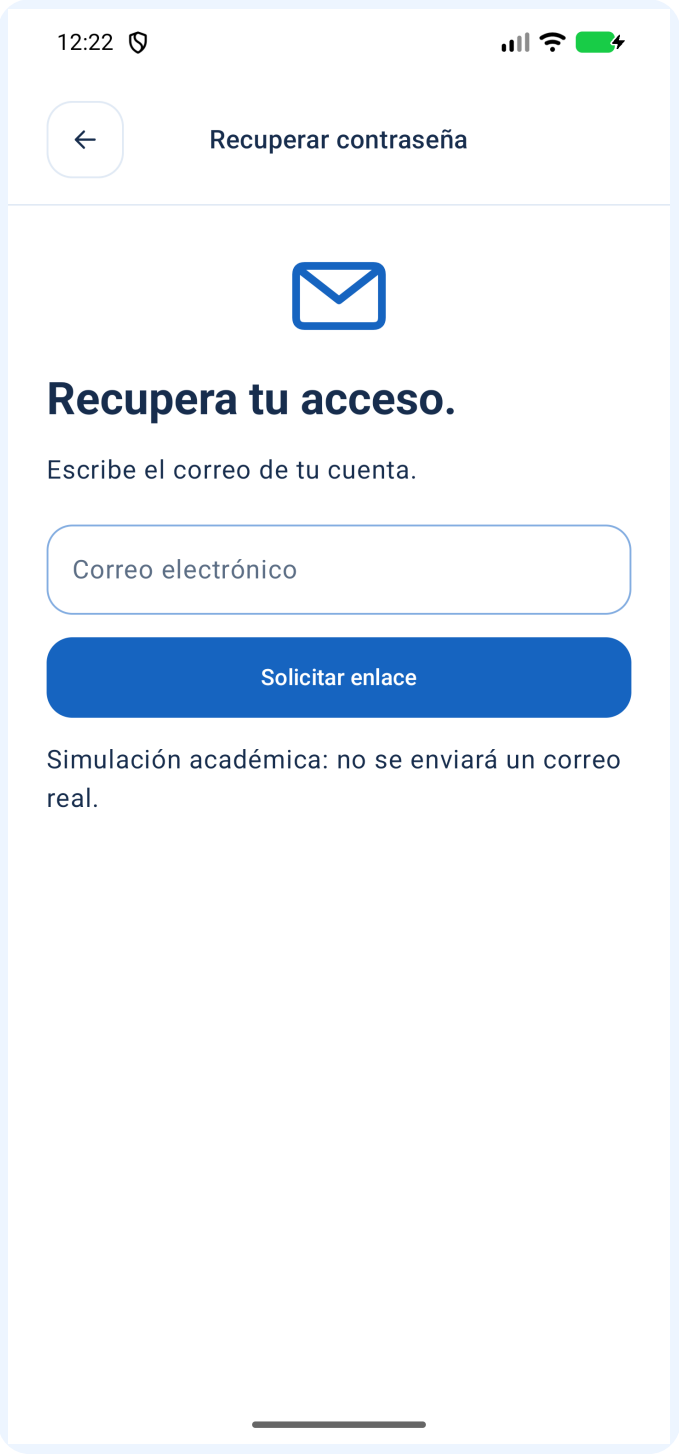
No se mantiene una sesión entre procesos. Google queda identificado como integración externa pendiente.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 01-login.png

03 · EVIDENCIAS FUNCIONALES

RF-04 · Solicitar recuperación



IMPLEMENTACIÓN

Usuario y moderador pueden introducir su correo para iniciar la recuperación. Se valida el formato.

RESPECTO AL PROTOTIPO

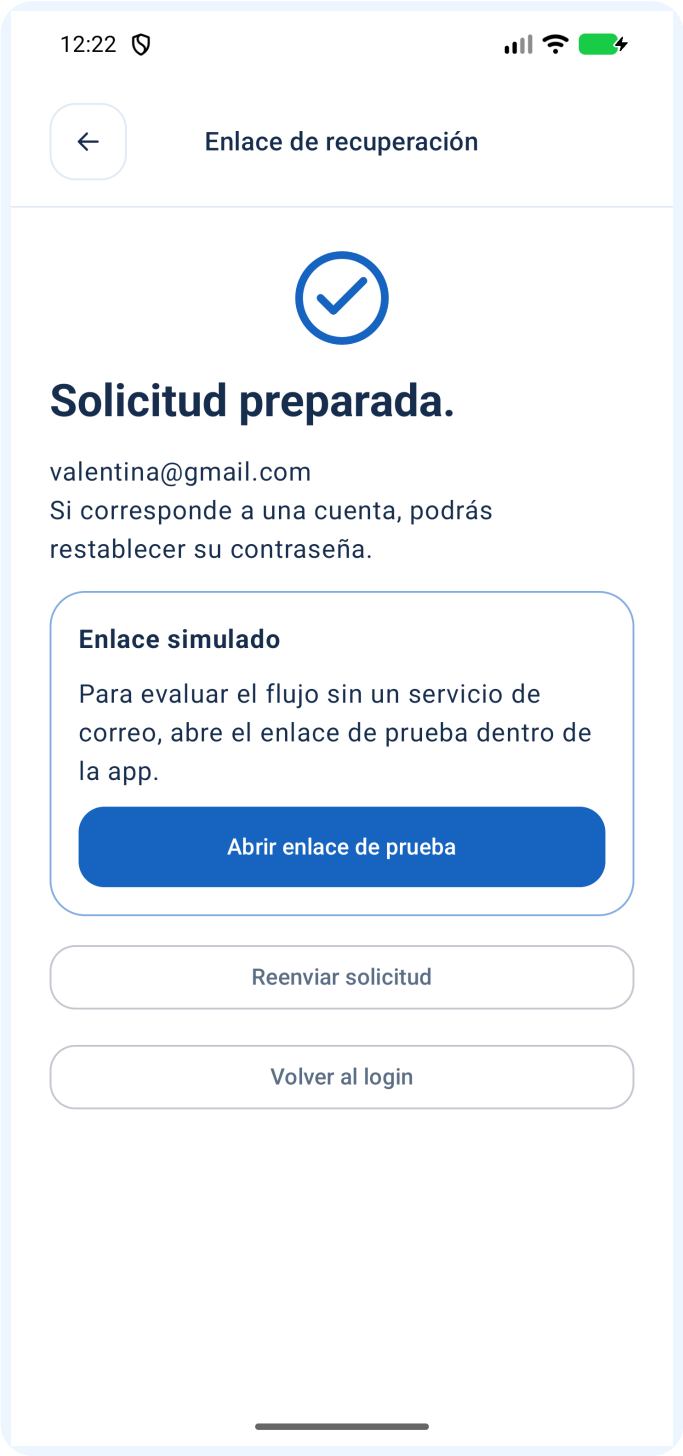
No hay envío real de correo; la consigna no requiere sesiones ni servicios externos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 05-recovery.png

03 · EVIDENCIAS FUNCIONALES

RF-05 · Enlace de recuperación



IMPLEMENTACIÓN

La pantalla intermedia permite abrir el enlace de prueba o repetir la solicitud.

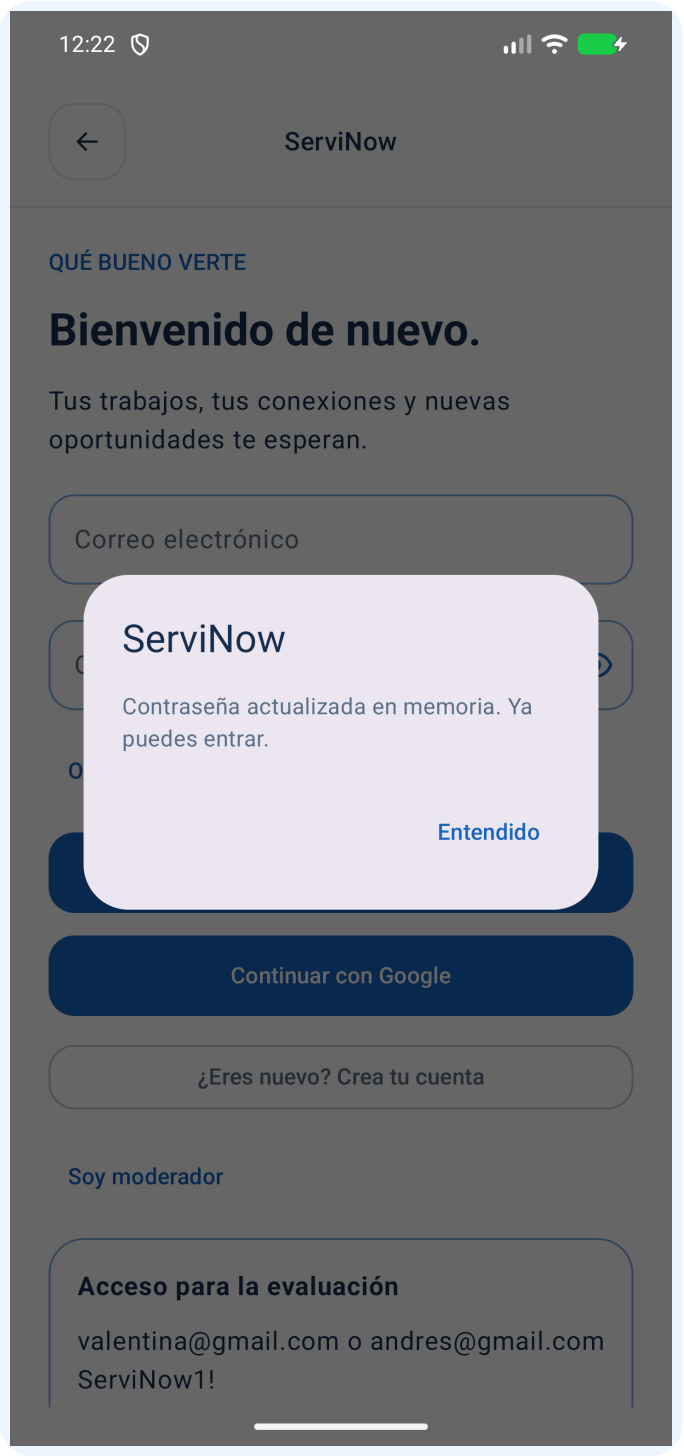
RESPECTO AL PROTOTIPO

El texto explicita la simulación para no afirmar que se envió un mensaje real.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 06-sent.png

RF-06 · Cambiar contraseña



IMPLEMENTACIÓN

El flujo valida longitud, mayúscula, número, carácter especial y confirmación. La nueva contraseña permite entrar y la anterior deja de funcionar.

RESPECTO AL PROTOTIPO

La actualización afecta únicamente la cuenta de prueba en memoria. El enlace se consume al usarlo.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 07-password-saved.png

RF-07 · Feed de publicaciones



IMPLEMENTACIÓN

Muestra trabajos públicos y permite abrir detalles. Los pendientes y rechazados no forman parte del feed.

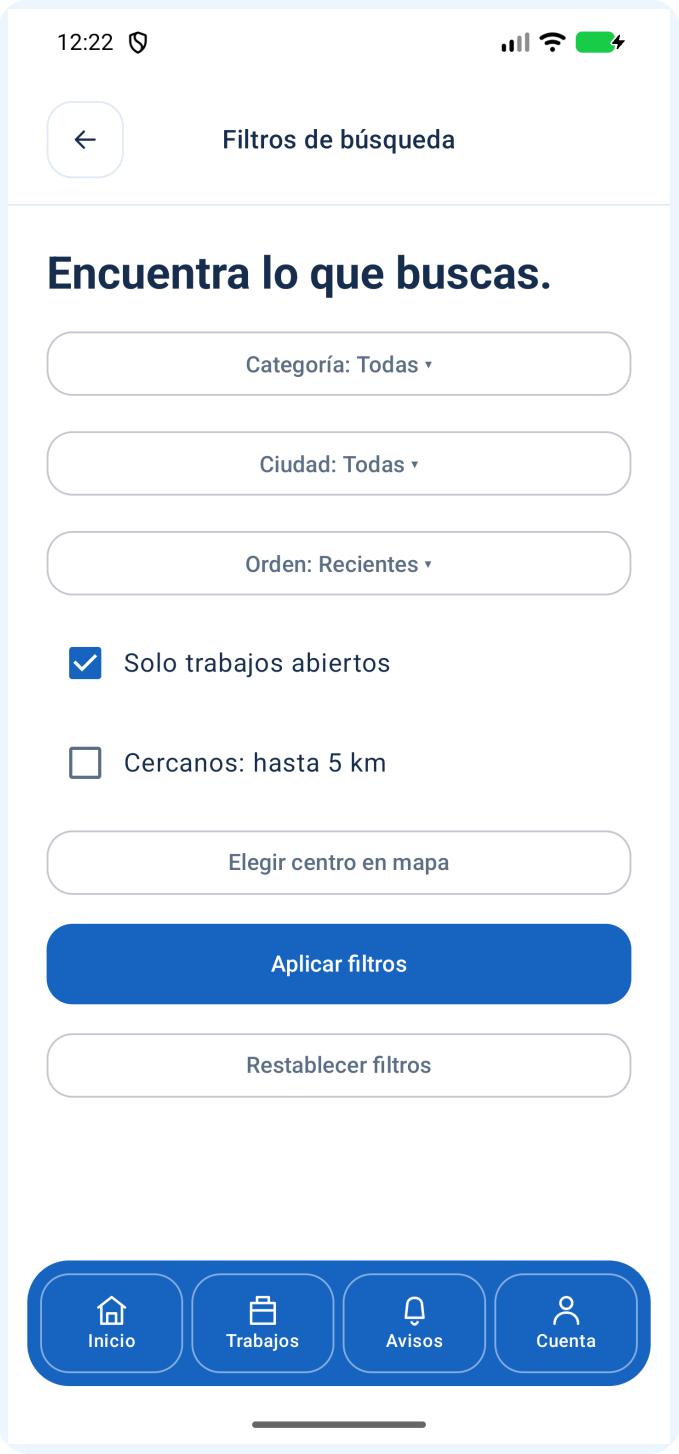
RESPECTO AL PROTOTIPO

Se mantiene el diseño blanco y azul; los datos del feed ahora comparten el repositorio de publicaciones.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 08-home.png

RF-08 · Filtrar por categoría y ubicación



IMPLEMENTACIÓN

Permite elegir categoría, ciudad, orden por relevancia o fecha, solo abiertos y cercanía a un punto de referencia.

RESPECTO AL PROTOTIPO

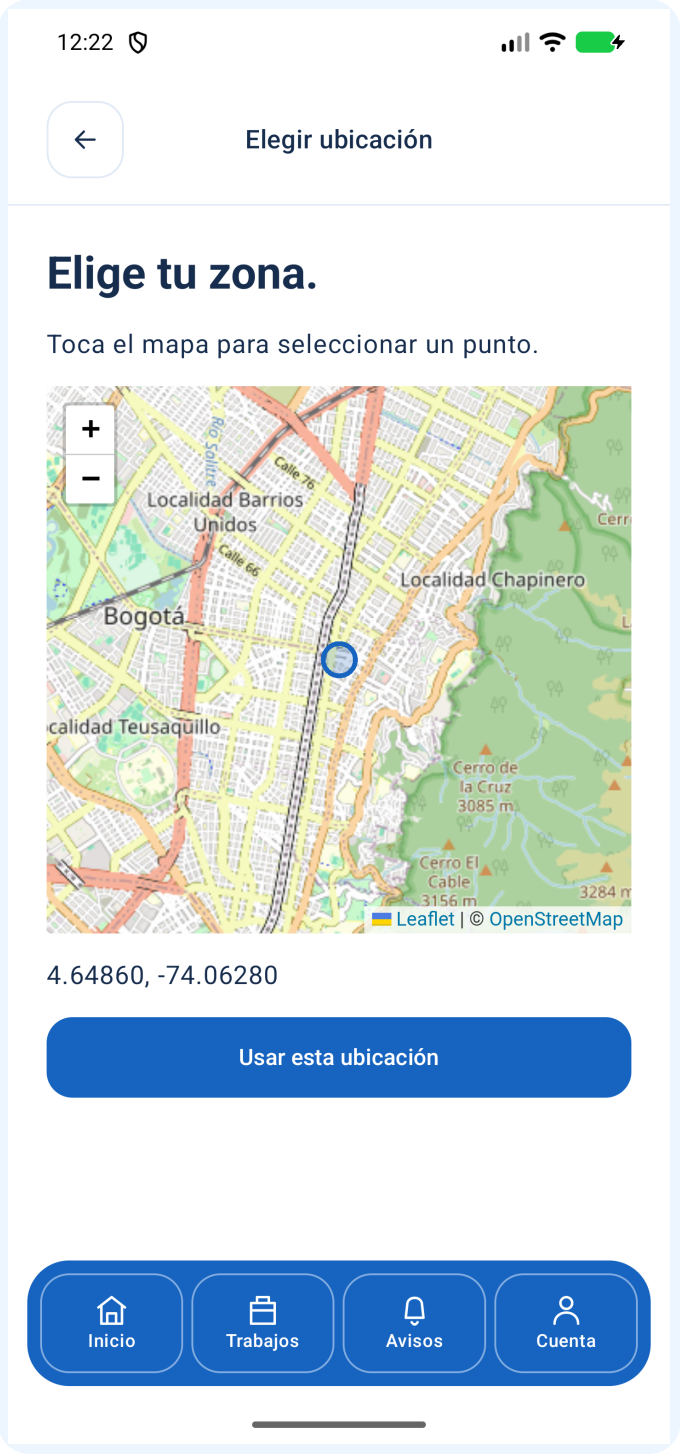
Los filtros funcionan con las coordenadas de los datos de prueba. La cercanía usa una distancia de hasta 5 km.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 09-filters.png

03 · EVIDENCIAS FUNCIONALES

RF-09 · Mapa opcional y selección de zona



IMPLEMENTACIÓN

El mapa permite seleccionar el centro de búsqueda y confirmar las coordenadas.

RESPECTO AL PROTOTIPO

Se conserva la función desarrollada previamente, aunque está exenta en esta fase. Necesita internet para descargar los mosaicos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 10-location.png

03 · EVIDENCIAS FUNCIONALES

RF-10 · Detalle completo de una publicación



IMPLEMENTACIÓN

Presenta título, categoría, estado, descripción, presupuesto, ciudad, coordenadas, imagen de ejemplo y autor.

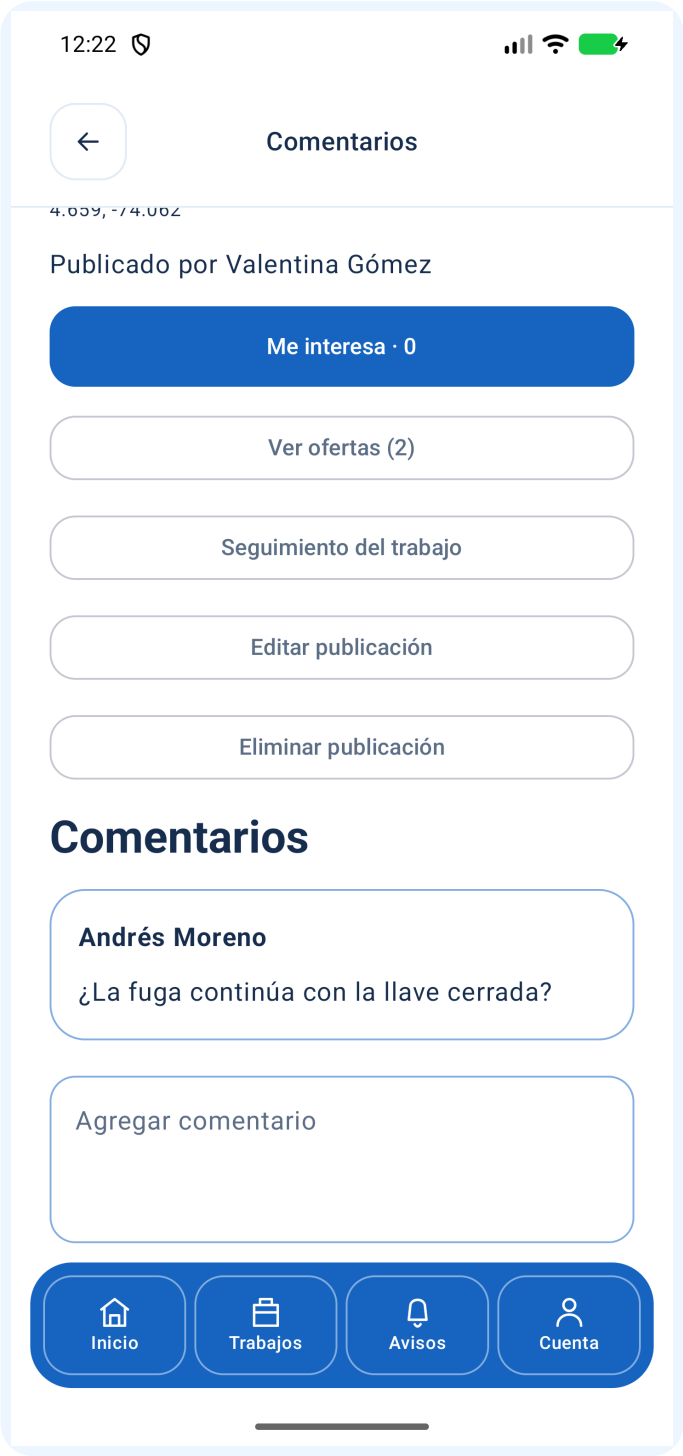
RESPECTO AL PROTOTIPO

Las fotos remotas se sustituyen por una ilustración identificada como ejemplo; el detalle conserva la información requerida.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 11-detail.png

RF-11 · Comentar y aportar información



IMPLEMENTACIÓN

Los comentarios se agregan a la publicación, indican autor y quedan visibles al cambiar de usuario. Un comentario ajeno genera un aviso al autor.

RESPECTO AL PROTOTIPO

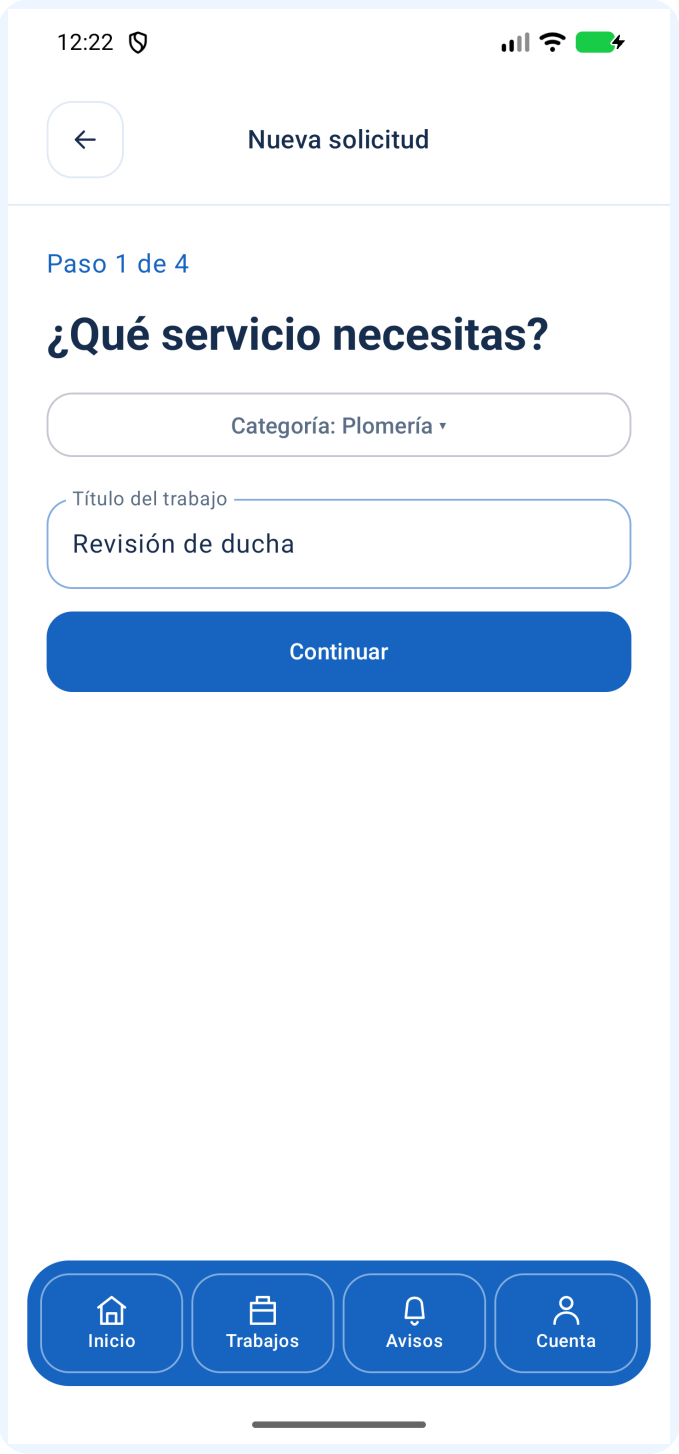
No se usa chat externo ni entrega en tiempo real. Los comentarios se guardan en el repositorio en memoria.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 12-comments.png

03 · EVIDENCIAS FUNCIONALES

RF-12 · Crear una solicitud



IMPLEMENTACIÓN

El usuario selecciona categoría y título. Los pasos siguientes recogen descripción, presupuesto y ubicación de referencia.

RESPECTO AL PROTOTIPO

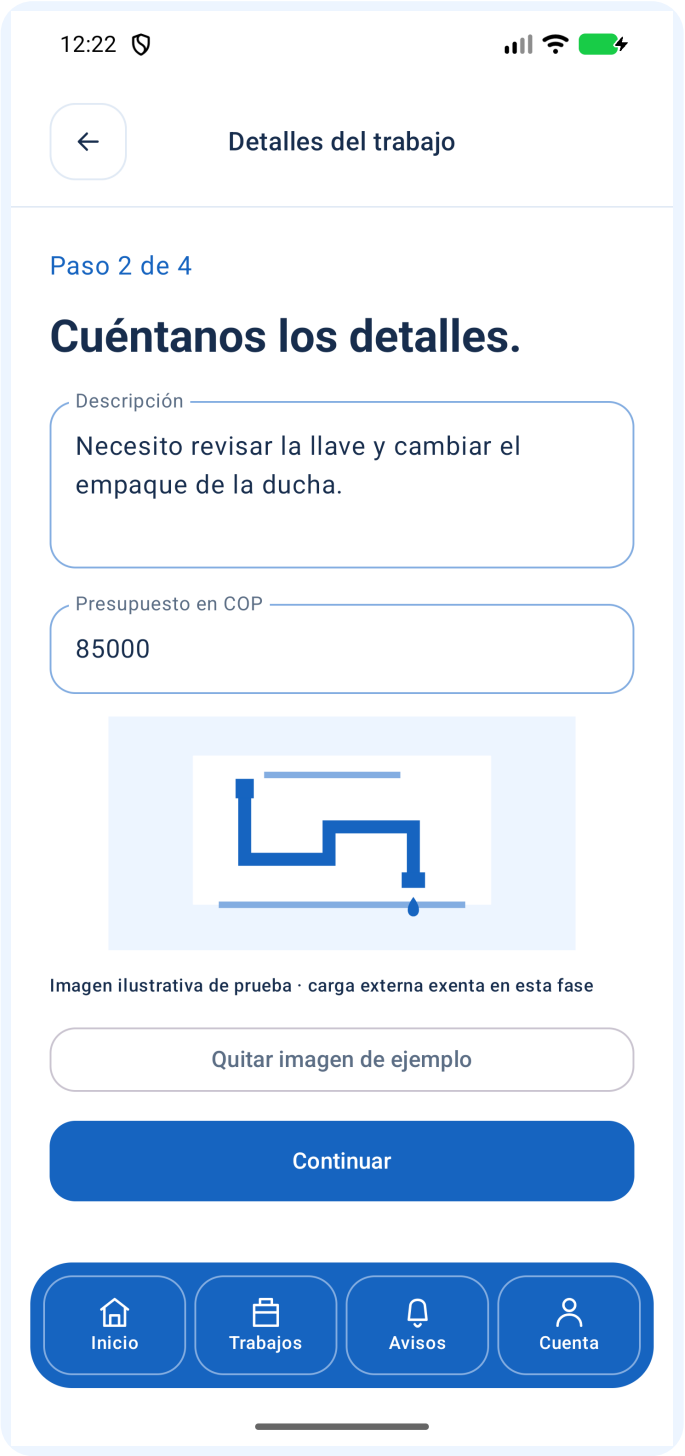
Se conserva el formulario por pasos y se añaden validaciones antes de avanzar.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 13-create.png

03 · EVIDENCIAS FUNCIONALES

RF-13 · Datos e imagen de la publicación



IMPLEMENTACIÓN

La descripción y el presupuesto se validan. Se selecciona una imagen ilustrativa para poder revisar la publicación completa.

RESPECTO AL PROTOTIPO

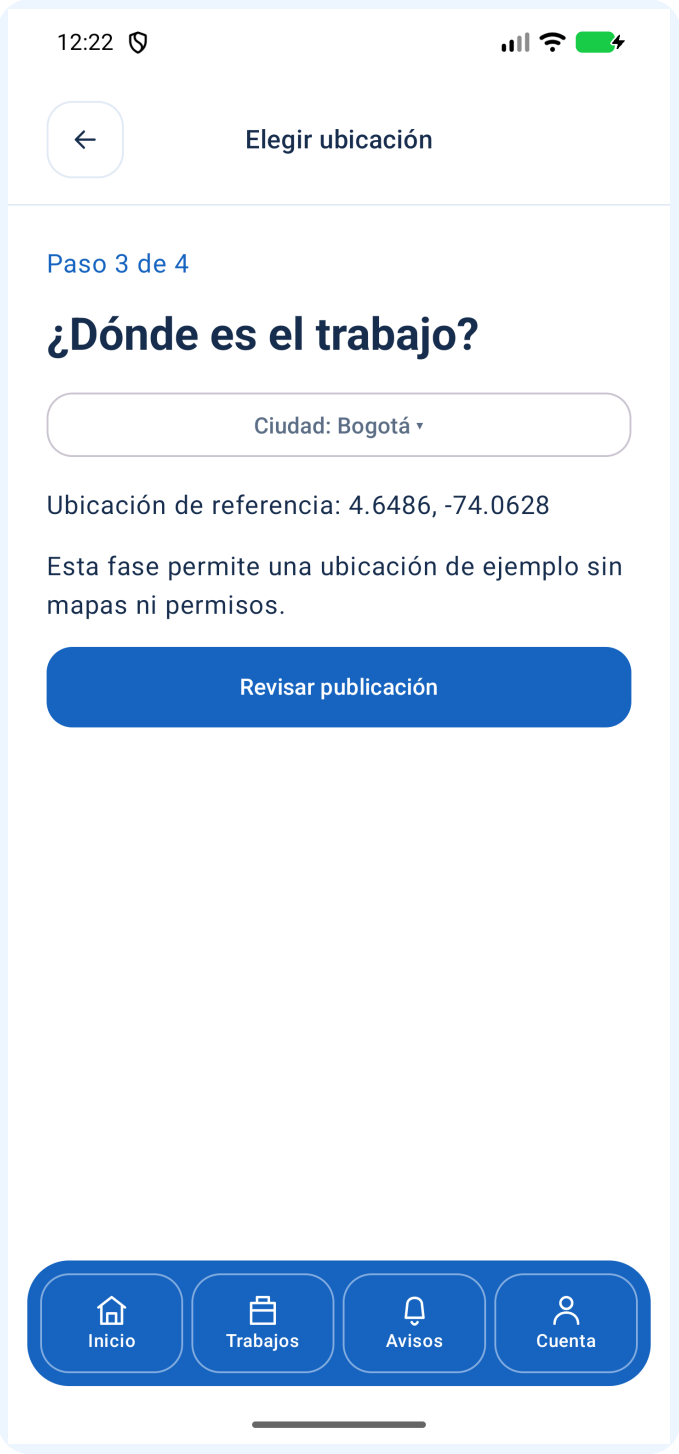
La carga de imágenes está exenta. El botón de ejemplo no sube archivos ni simula un servicio de almacenamiento.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 14-create-details.png

03 · EVIDENCIAS FUNCIONALES

RF-14 · Ubicación de la solicitud



IMPLEMENTACIÓN

La ciudad seleccionada asigna coordenadas de referencia al trabajo.

RESPECTO AL PROTOTIPO

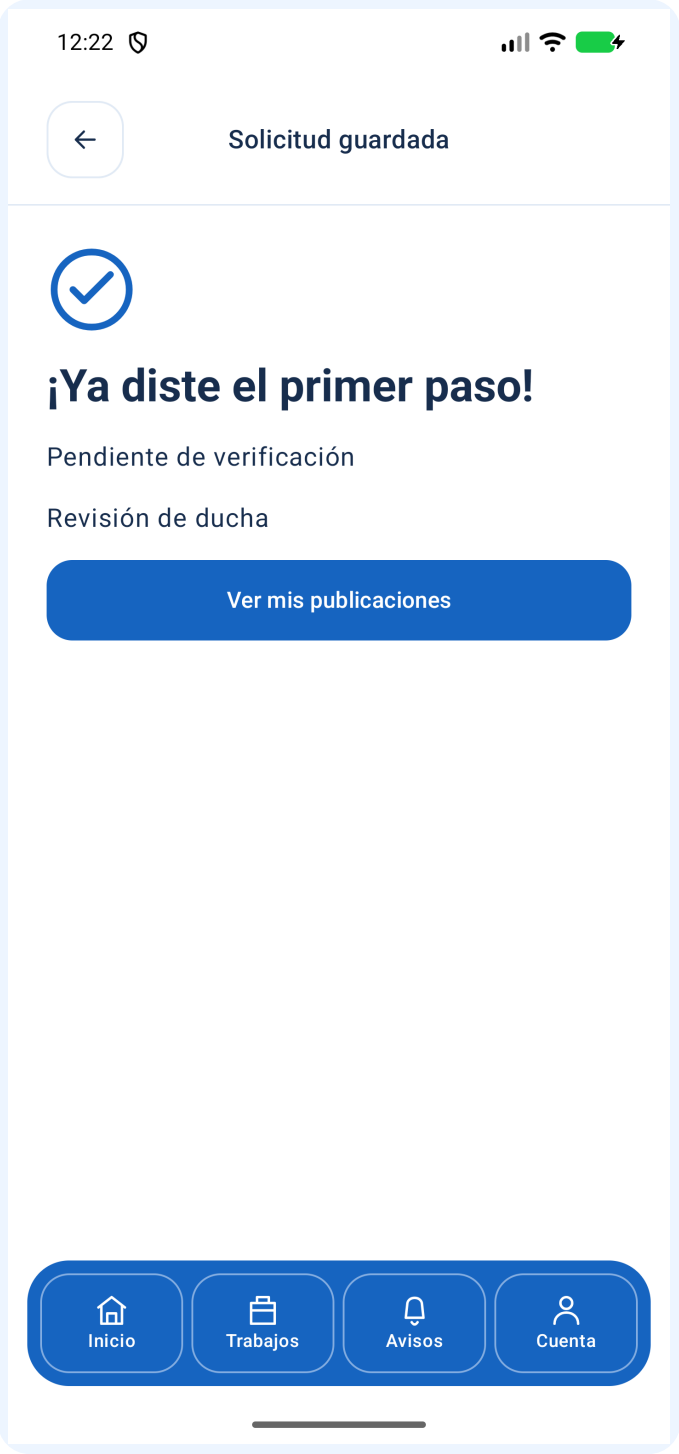
Se evita exigir permisos o un mapa en el registro de publicaciones, según la consigna de la fase dos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 15-post-location.png

03 · EVIDENCIAS FUNCIONALES

RF-15 · Creación sin verificar



IMPLEMENTACIÓN

Al guardar, la publicación queda pendiente, genera un aviso y no se hace pública hasta la verificación.

RESPECTO AL PROTOTIPO

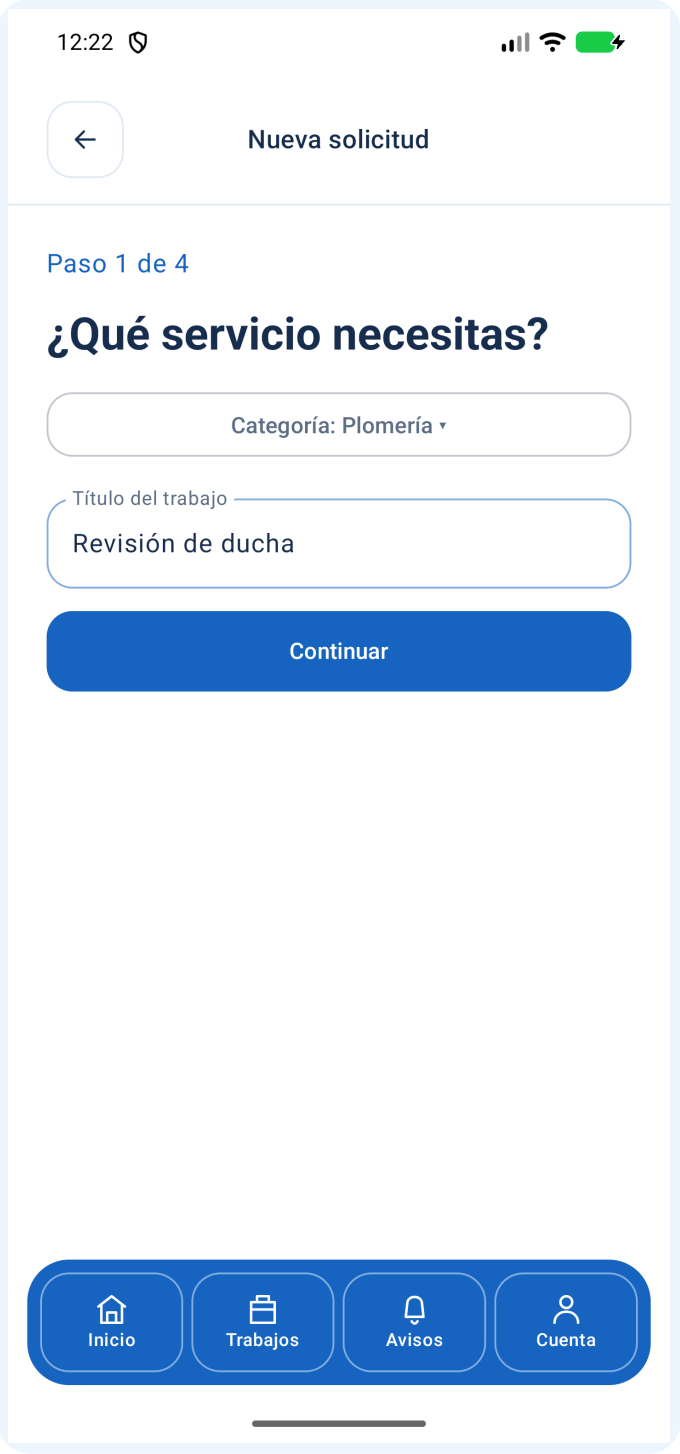
El cambio de estado es funcional en memoria y puede comprobarse con la cuenta de moderador.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 17-submitted.png

03 · EVIDENCIAS FUNCIONALES

RF-16 · Consultar y editar publicaciones propias



IMPLEMENTACIÓN

El autor puede editar trabajos pendientes, verificados o rechazados. Guardar una edición devuelve la solicitud a revisión.

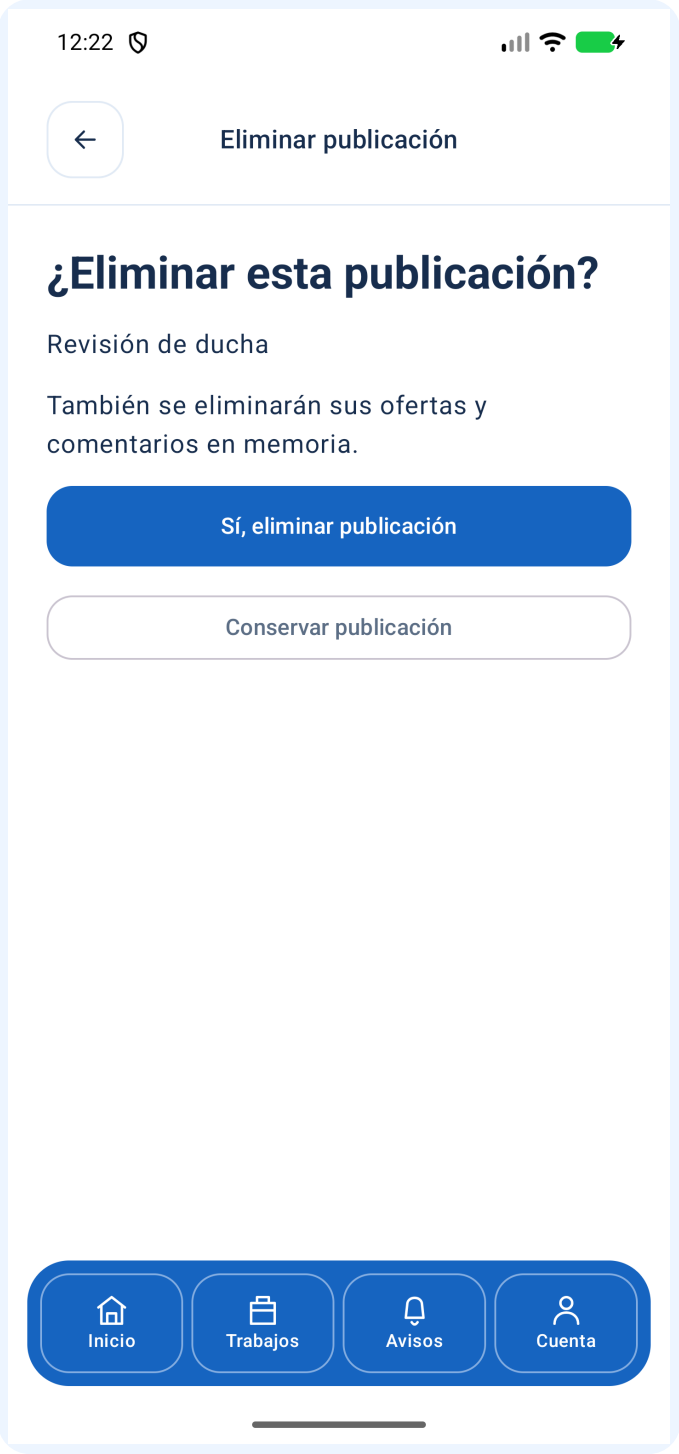
RESPECTO AL PROTOTIPO

Los trabajos asignados o finalizados no se editan para proteger el acuerdo existente.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 19-edit.png

RF-17 · Eliminar una publicación



IMPLEMENTACIÓN

Solo el autor puede eliminar. La pantalla confirma la intención y se eliminan también sus ofertas, comentarios y reseñas asociadas.

RESPECTO AL PROTOTIPO

Se aplica al repositorio en memoria; no afecta archivos originales del dispositivo.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 26-delete-post.png

RF-18 · Rechazo visible para el autor



IMPLEMENTACIÓN

El autor ve el motivo de rechazo y puede corregir la solicitud. La publicación rechazada queda fuera del feed público.

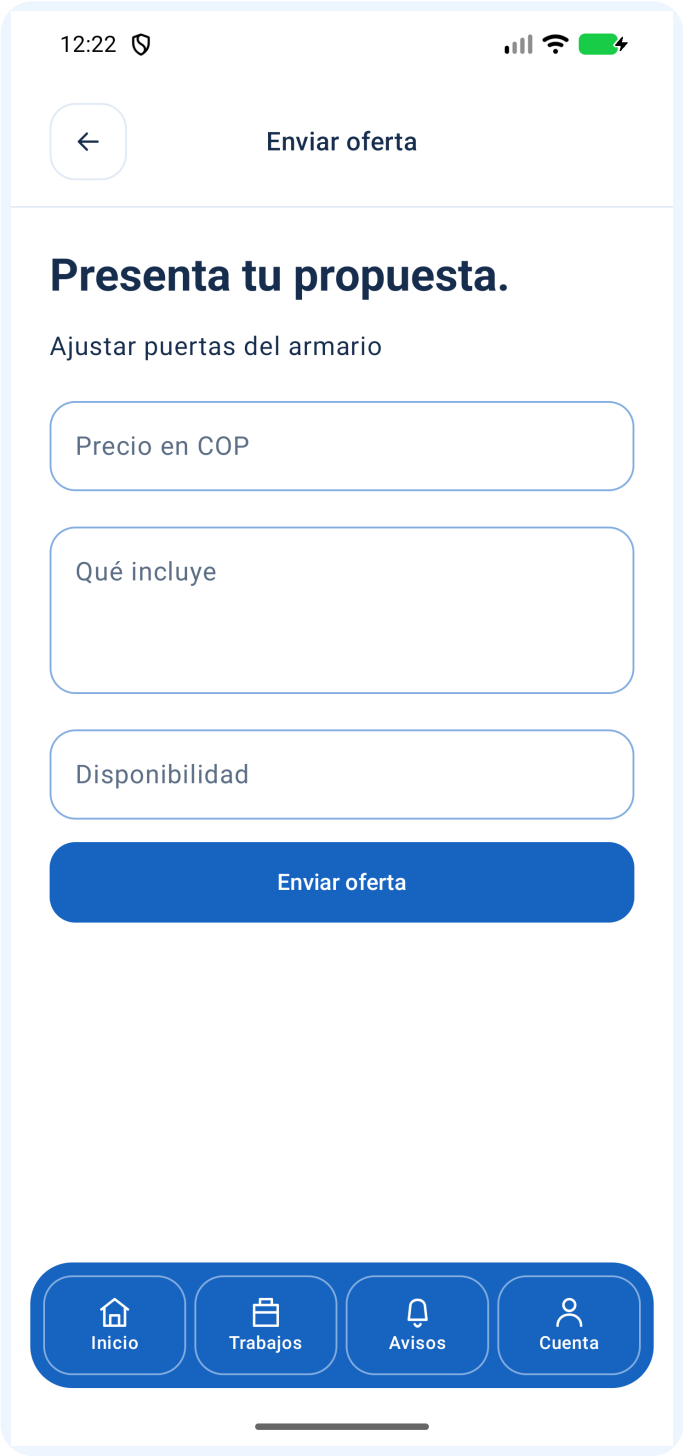
RESPECTO AL PROTOTIPO

El motivo procede del estado de la publicación y no de un texto fijo del prototipo.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 20-rejected.png

RF-19 · Ofertar por un trabajo



IMPLEMENTACIÓN

El trabajador introduce precio, alcance y disponibilidad. Solo se aceptan ofertas en trabajos verificados, ajenos y sin una oferta previa del mismo usuario.

RESPECTO AL PROTOTIPO

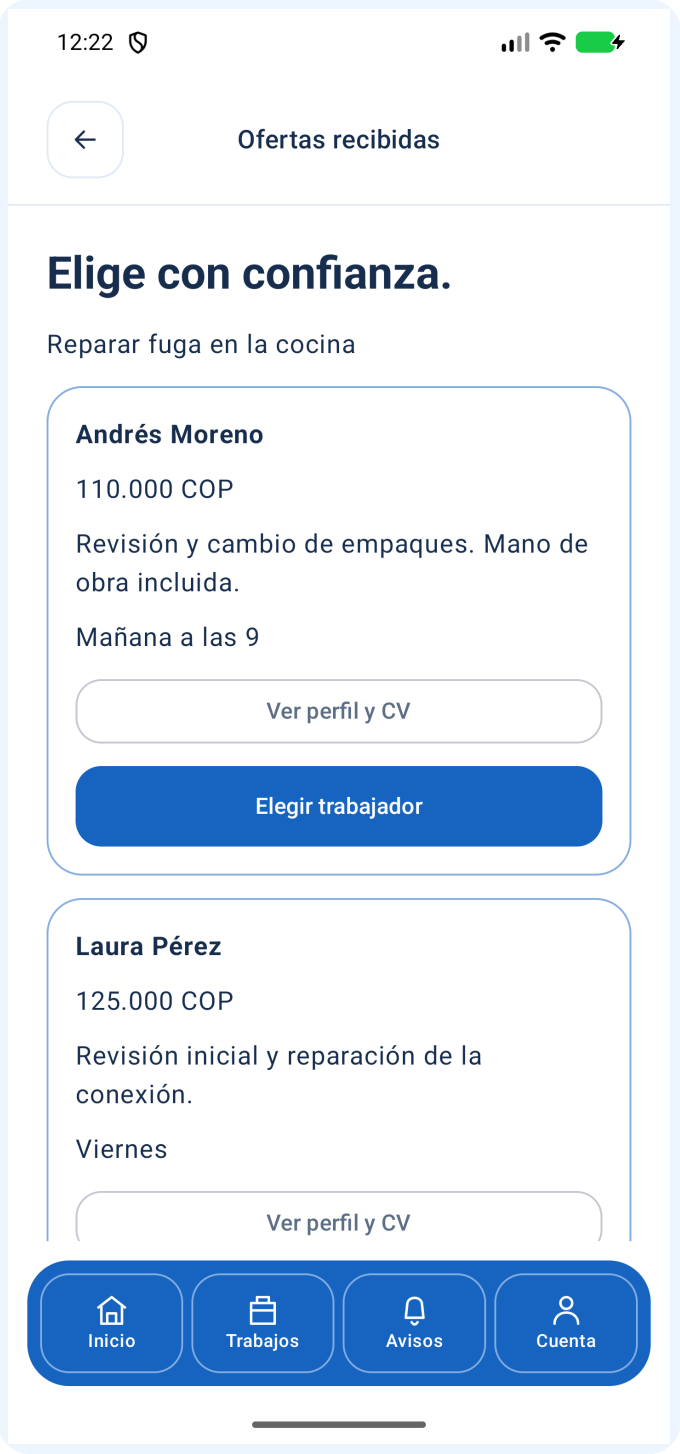
La oferta se guarda en memoria y genera un aviso al solicitante.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 28-offer.png

03 · EVIDENCIAS FUNCIONALES

RF-20 · Consultar ofertas recibidas



IMPLEMENTACIÓN

El autor puede comparar precio, propuesta, disponibilidad y perfil de cada trabajador.

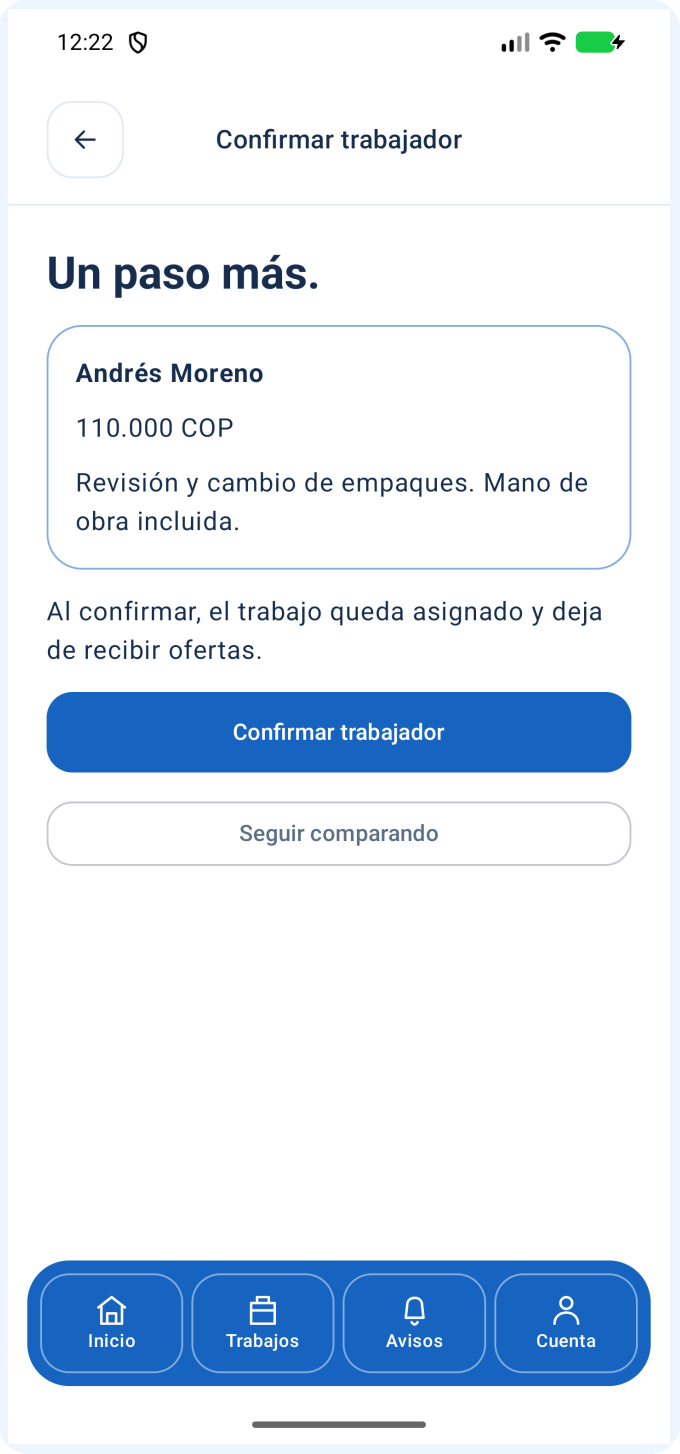
RESPECTO AL PROTOTIPO

Las tarjetas son únicas, sin recuadros anidados alrededor de la persona.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 21-offers.png

RF-21 · Aceptar una oferta



IMPLEMENTACIÓN

Solo el autor puede confirmar una oferta. El trabajo pasa a en progreso y deja de aceptar nuevas ofertas. El trabajador recibe un aviso.

RESPECTO AL PROTOTIPO

La selección se vuelve exclusiva; no se puede asignar un segundo trabajador al mismo trabajo.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 22-confirm.png

RF-22 · Seguimiento del trabajo



IMPLEMENTACIÓN

Se visualizan el estado y el trabajador asignado, con acceso al perfil y a la conversación.

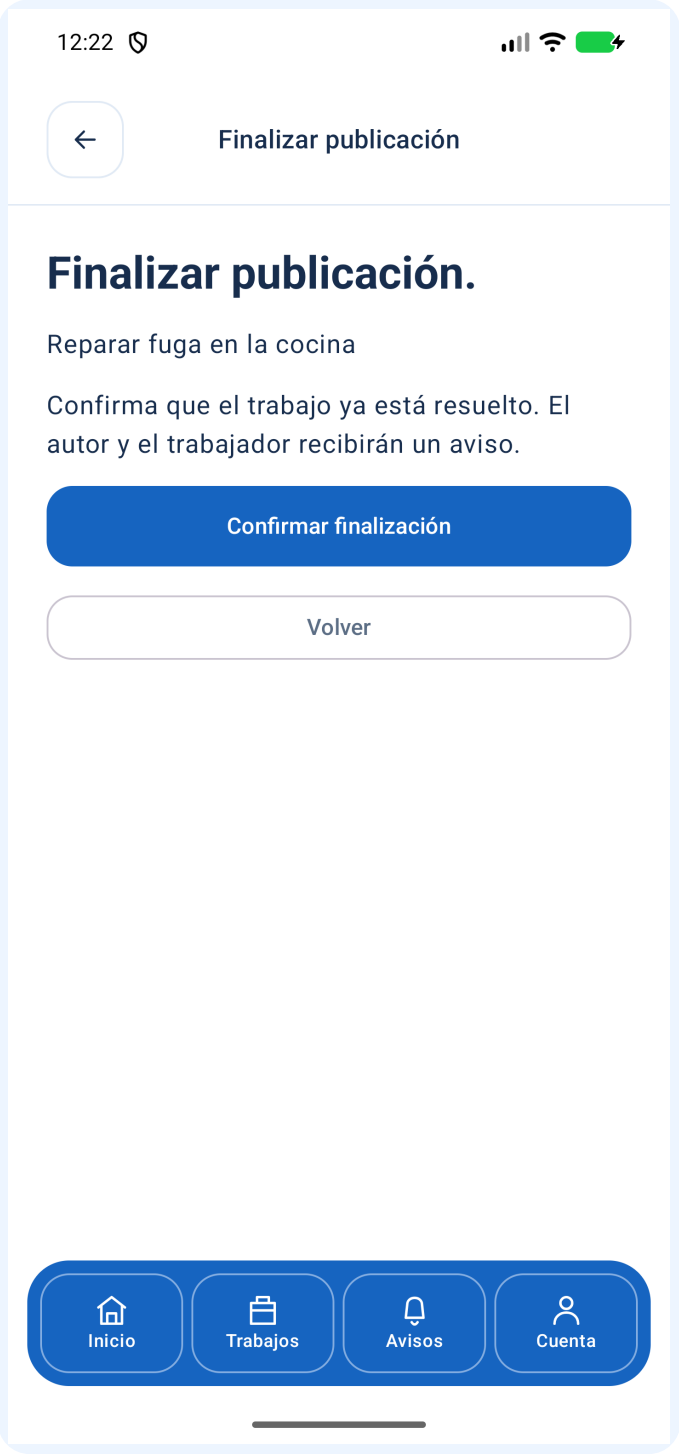
RESPECTO AL PROTOTIPO

Las etapas reflejan el estado del repositorio; no constituyen rastreo en tiempo real.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 23-tracking.png

RF-23 · Finalizar por el autor



IMPLEMENTACIÓN

El autor puede confirmar que su publicación está resuelta. Se actualiza el estado y se avisa al autor y al trabajador asignado.

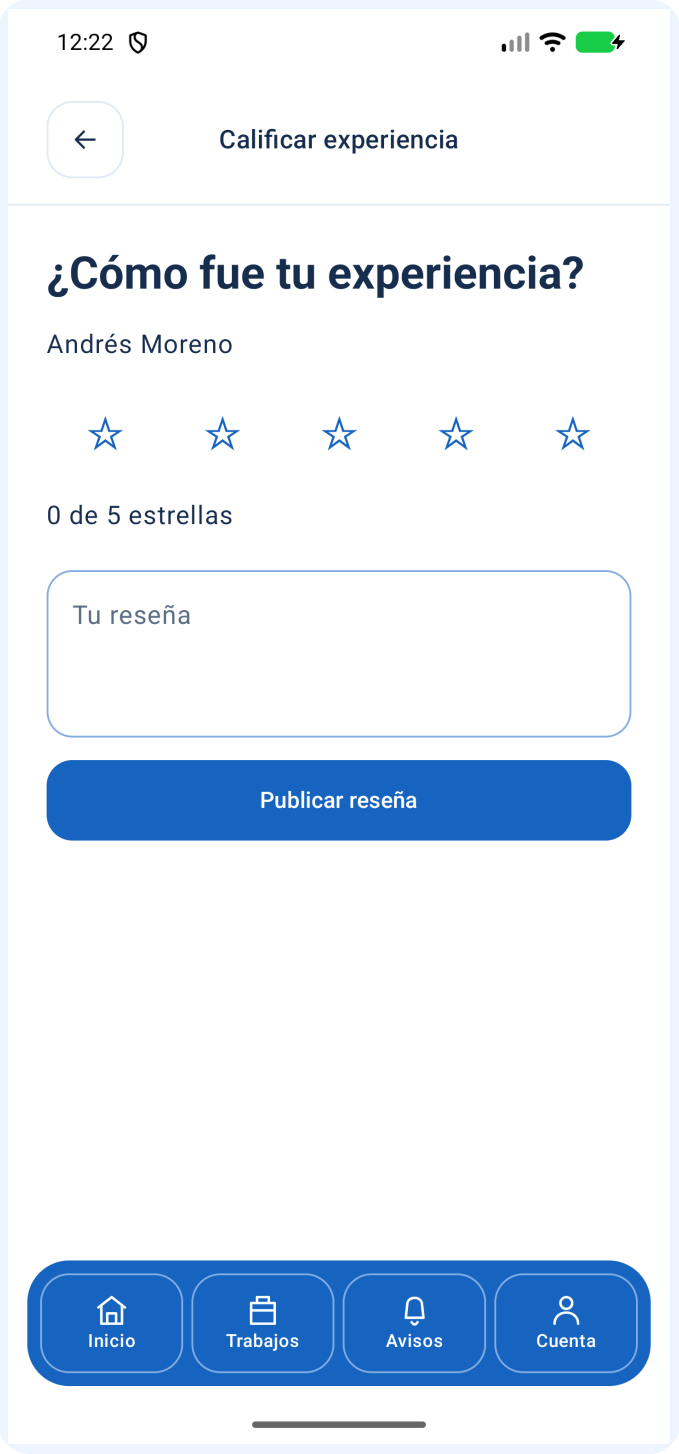
RESPECTO AL PROTOTIPO

La finalización habilita la reseña cuando existe un trabajador asignado.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 24-finish.png

RF-24 · Calificar y reseñar al trabajador



IMPLEMENTACIÓN

El solicitante elige de una a cinco estrellas y escribe una reseña tras finalizar. Solo se admite una reseña por trabajo.

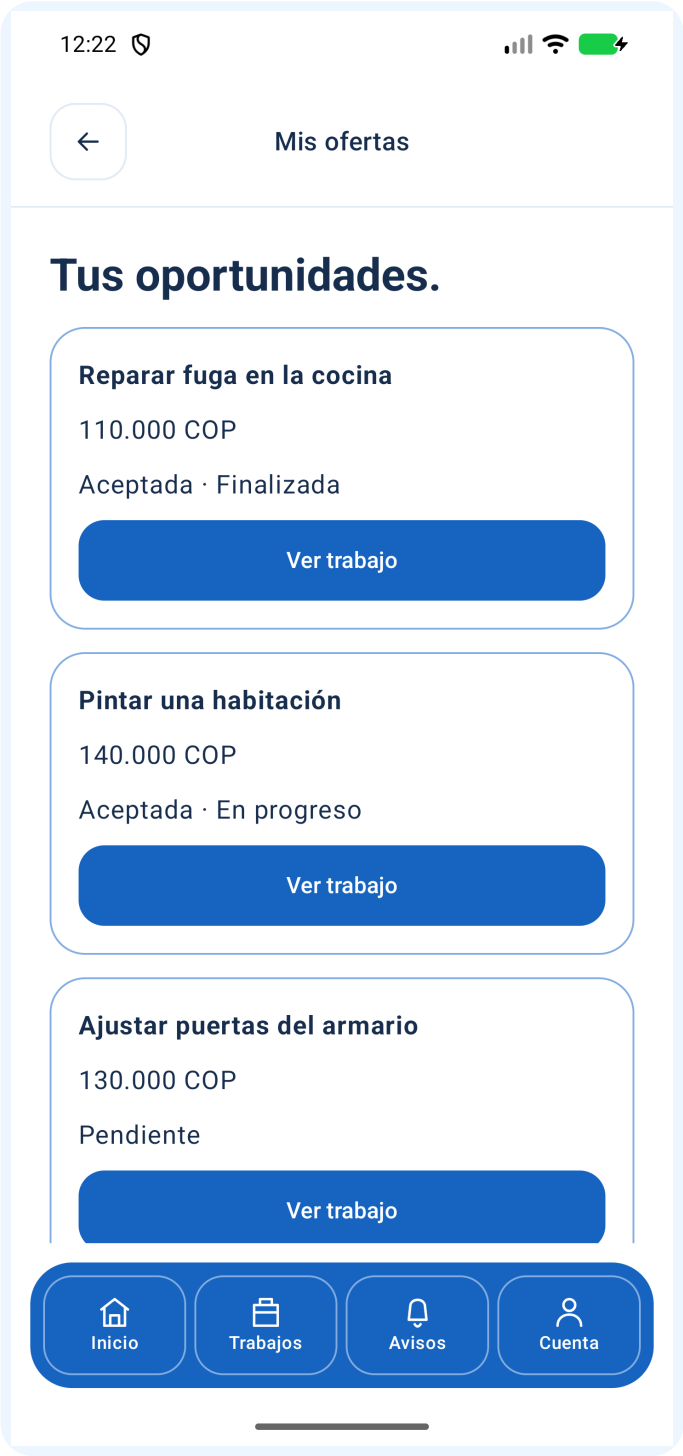
RESPECTO AL PROTOTIPO

La puntuación del perfil se calcula con las reseñas del repositorio.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 25-review.png

RF-25 · Panel y ofertas del trabajador



IMPLEMENTACIÓN

Permite consultar propuestas enviadas, precios y si están pendientes o aceptadas. Desde el panel se abren los trabajos asignados.

RESPECTO AL PROTOTIPO

Solicitante y trabajador son formas de uso del mismo rol Usuario, no permisos de moderación.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 29-my-offers.png

RF-26 · Perfil público e información profesional



IMPLEMENTACIÓN

Muestra nombre, especialidad, presentación, experiencia, calificaciones y reseñas de la comunidad.

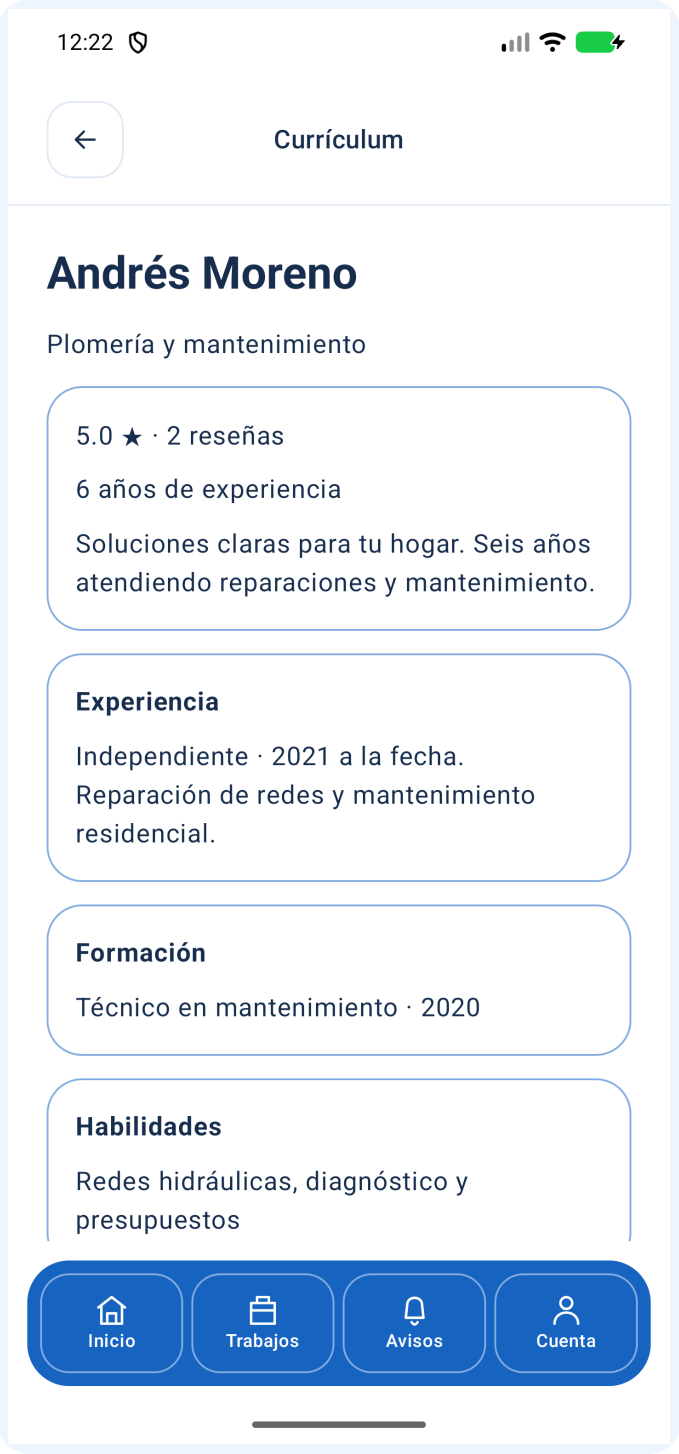
RESPECTO AL PROTOTIPO

Se conserva una tarjeta por bloque y el encabezado solo muestra campana en perfil/cuenta.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 30-profile.png

RF-27 · Consultar currículum



IMPLEMENTACIÓN

El currículum presenta experiencia laboral, formación y habilidades del trabajador.

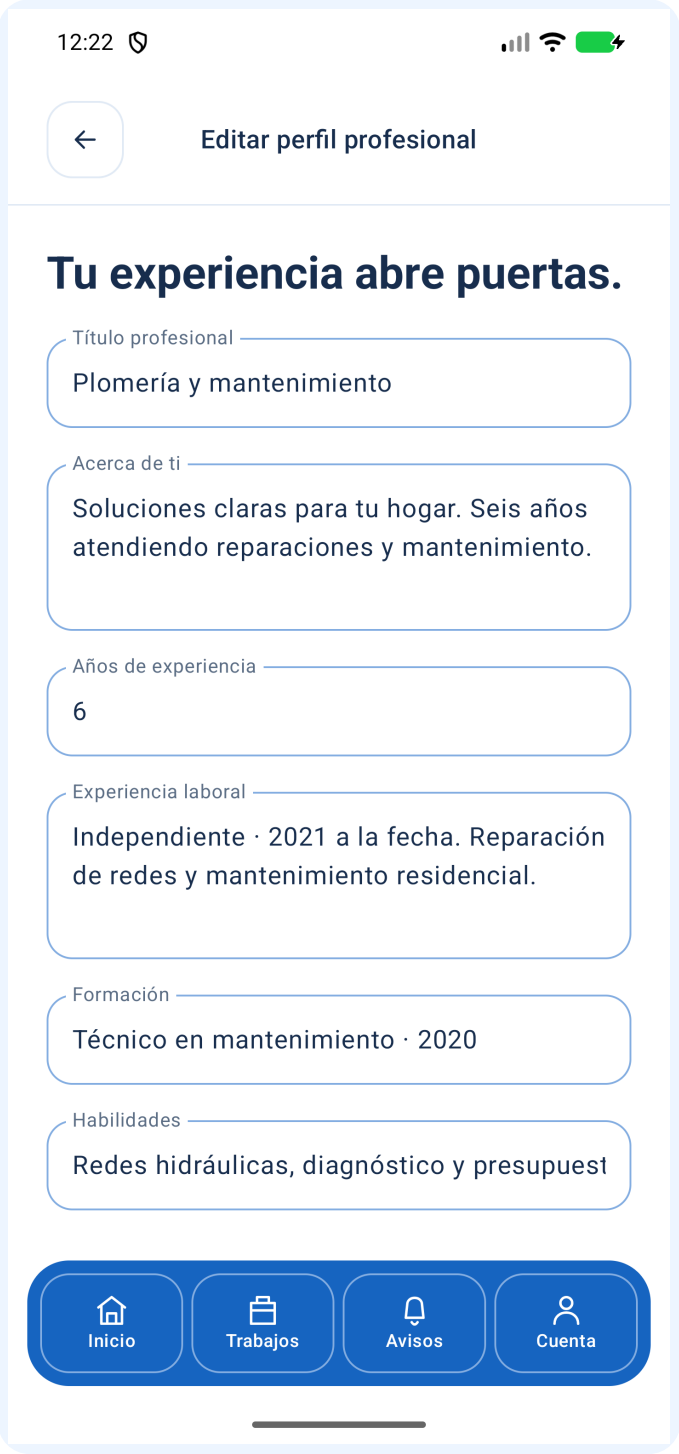
RESPECTO AL PROTOTIPO

Se genera con datos editables del perfil. No se exige subir un PDF ni gestionar permisos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 31-cv.png

RF-28 · Editar perfil profesional



IMPLEMENTACIÓN

El usuario modifica título, presentación, años de experiencia, trayectoria, formación y habilidades.

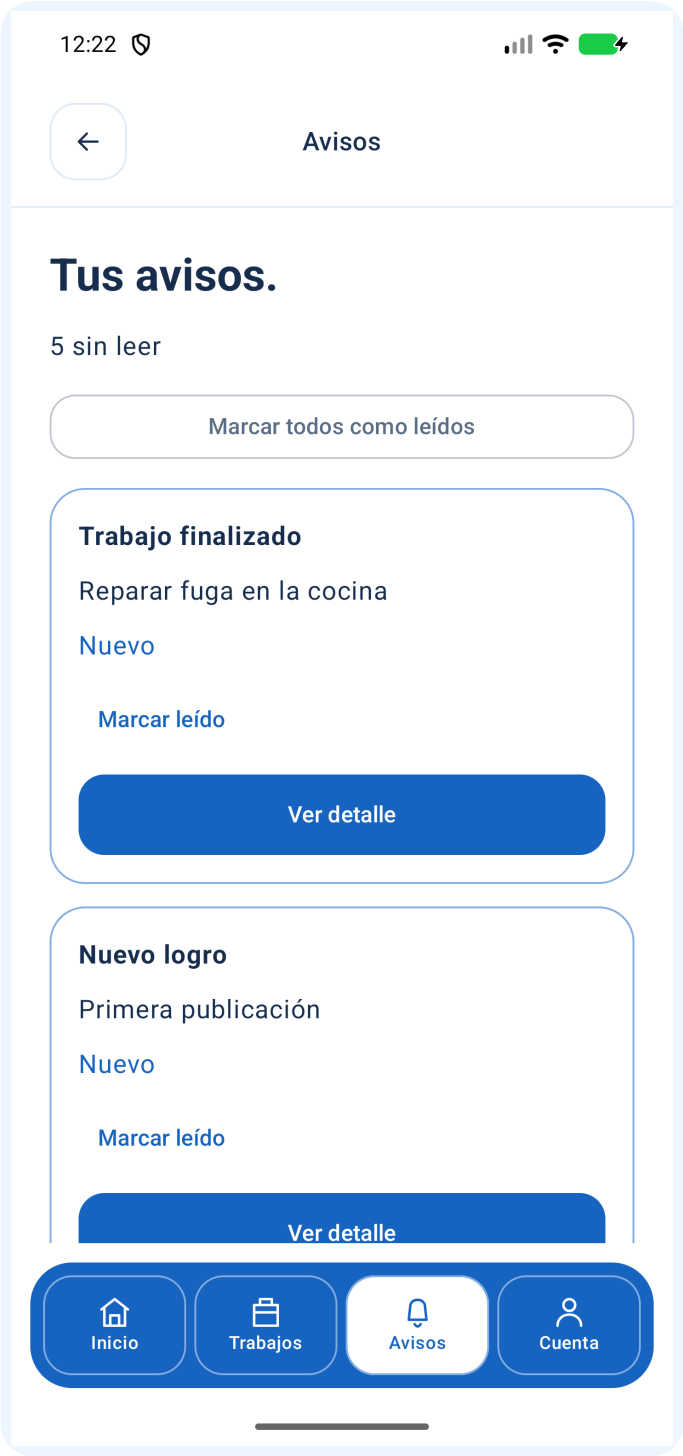
RESPECTO AL PROTOTIPO

Los cambios se reflejan en el perfil y currículum del mismo usuario en memoria.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 32-professional.png

RF-29 · Notificaciones leídas y no leídas



IMPLEMENTACIÓN

La lista muestra avisos propios y permite marcar uno o todos como leídos, revertir la lectura y abrir la publicación relacionada.

RESPECTO AL PROTOTIPO

Los avisos se almacenan en memoria, sin Firebase Cloud Messaging ni base de datos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 33-notifications.png

RF-30 · Estadísticas personales



IMPLEMENTACIÓN

Calcula publicaciones activas, finalizadas, pendientes y rechazadas; además muestra comentarios, votos recibidos, puntos y nivel.

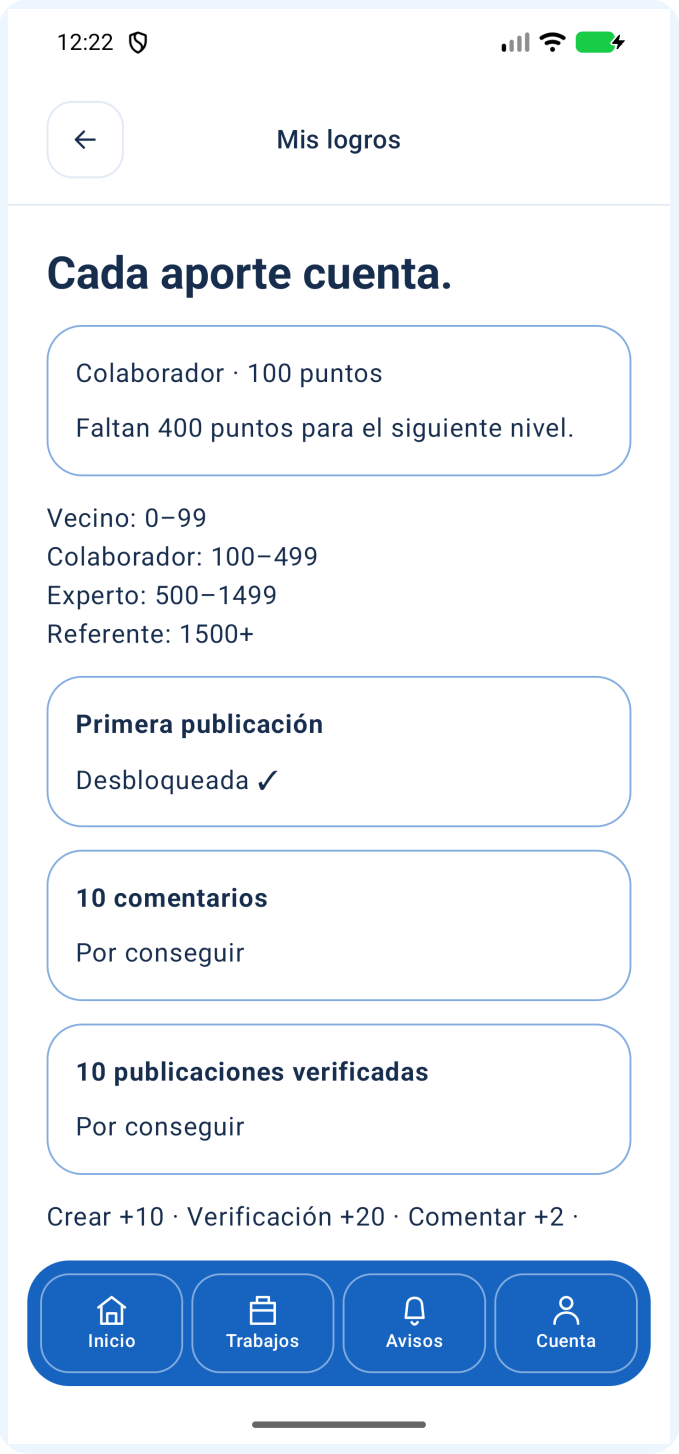
RESPECTO AL PROTOTIPO

Los contadores corresponden al usuario activo de la demostración y se recalculan desde el estado compartido.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 34-dashboard.png

RF-31 · Puntos y cuatro niveles



IMPLEMENTACIÓN

Crear otorga 10 puntos, verificar 20, comentar 2 y recibir un voto 1. Los niveles son Vecino, Colaborador, Experto y Referente.

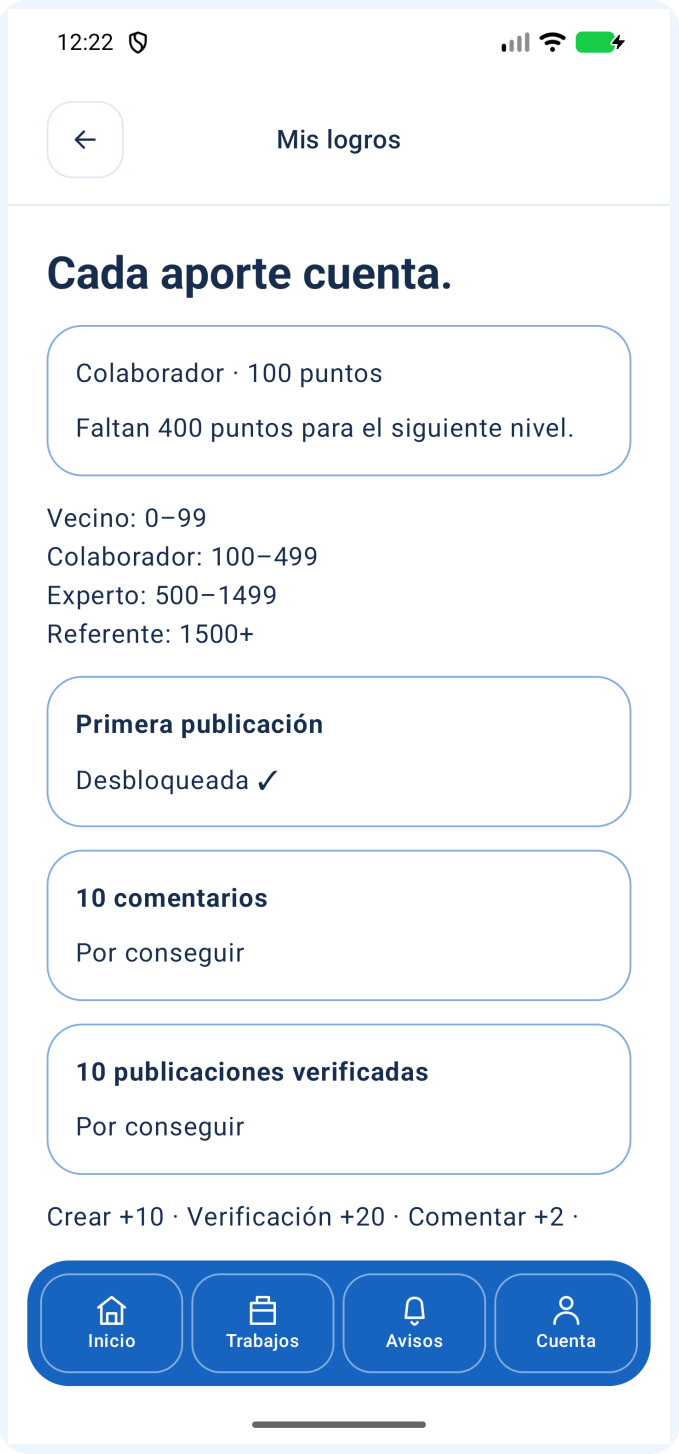
RESPECTO AL PROTOTIPO

Se evita otorgar puntos por auto-votos o por alternar repetidamente el mismo voto.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 35-achievements.png

RF-32 · Insignias por logros



IMPLEMENTACIÓN

Se desbloquean primera publicación, diez comentarios y diez publicaciones verificadas; el desbloqueo genera una notificación.

RESPECTO AL PROTOTIPO

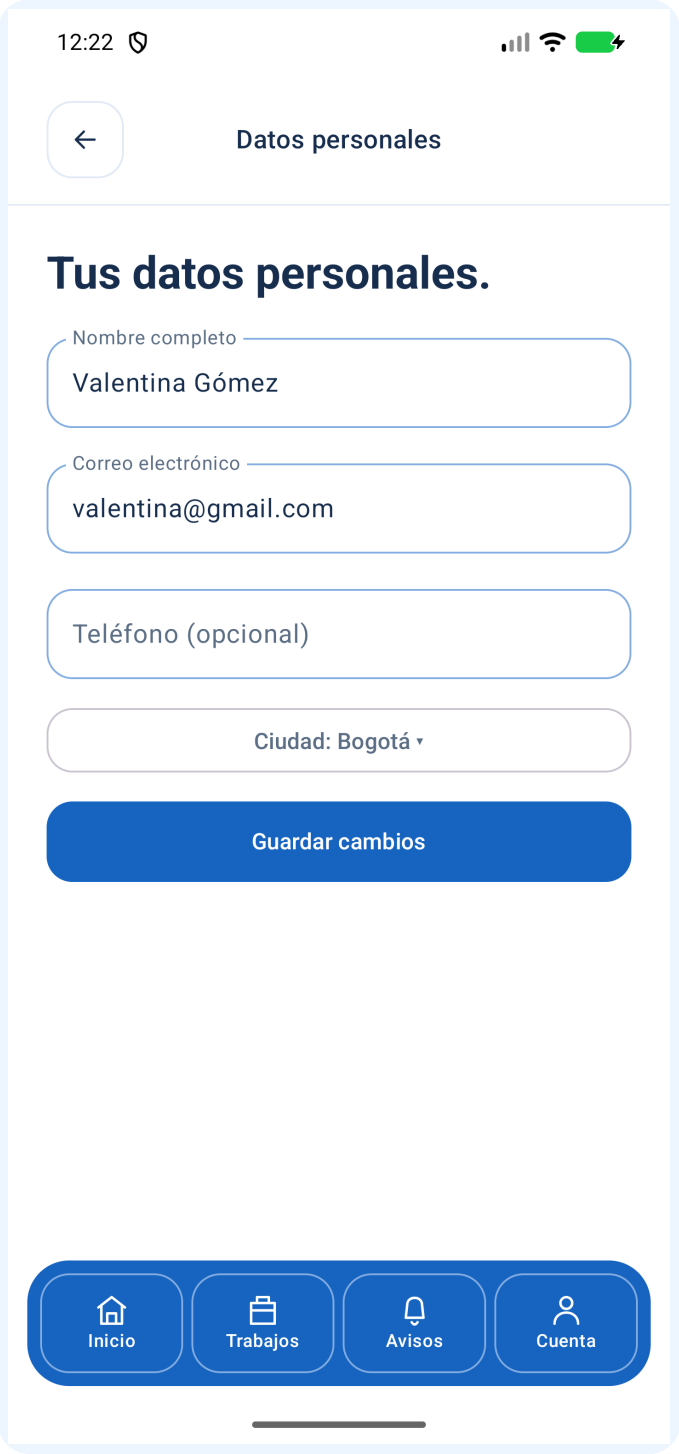
Se eligieron logros verificables en la demostración. El destacado mensual era un ejemplo opcional del planteamiento inicial.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 35-achievements.png

03 · EVIDENCIAS FUNCIONALES

RF-33 · Editar datos personales



IMPLEMENTACIÓN

El usuario cambia nombre, correo, teléfono y ciudad. Se valida el correo y se impiden duplicados; no puede cambiar su rol.

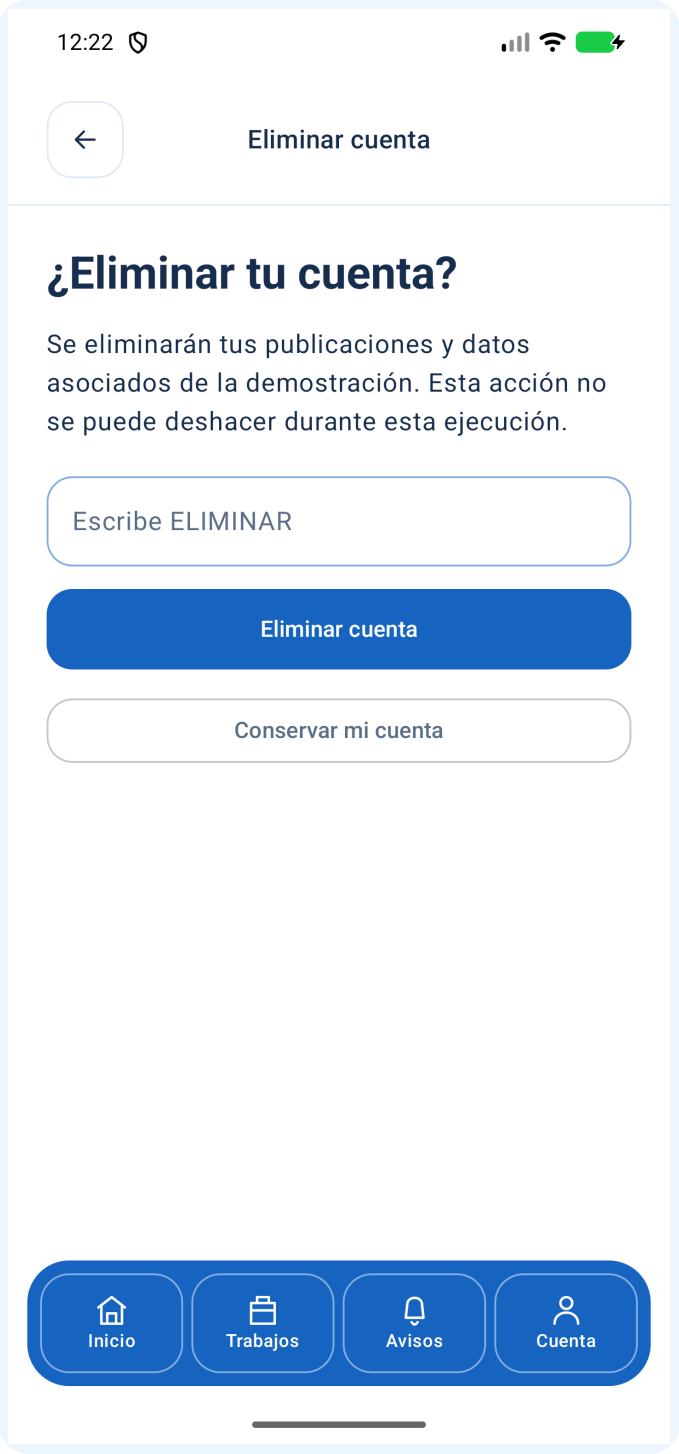
RESPECTO AL PROTOTIPO

La dirección se simplifica a ciudad en esta fase, que no exige mapa ni permisos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 37-edit-account.png

RF-34 · Eliminar cuenta



IMPLEMENTACIÓN

Se requiere escribir ELIMINAR. Se retiran el usuario y sus datos asociados de la demostración y se vuelve al acceso.

RESPECTO AL PROTOTIPO

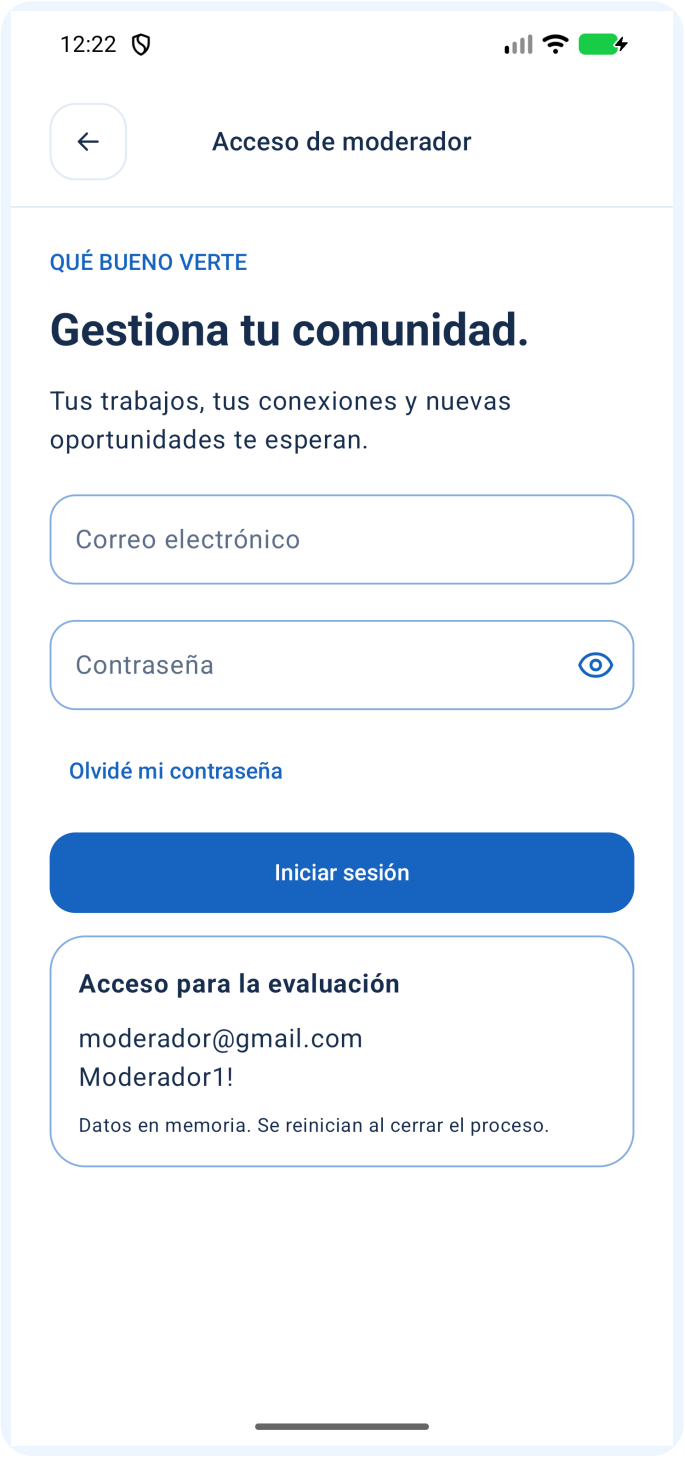
Las cuentas precargadas de moderador no se eliminan desde el menú de usuario.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 38-delete-account.png

03 · EVIDENCIAS FUNCIONALES

RF-35 · Acceso del moderador



IMPLEMENTACIÓN

Una cuenta precargada accede a la bandeja de revisión. Las credenciales de usuario no conceden privilegios de moderación.

RESPECTO AL PROTOTIPO

La comprobación se hace en memoria; no representa seguridad de un servidor ni una sesión persistente.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 39-mod-login.png

03 · EVIDENCIAS FUNCIONALES

RF-36 · Gestionar y verificar publicaciones



IMPLEMENTACIÓN

La bandeja permite filtrar por estado e inspeccionar una publicación. Verificarla la vuelve pública, otorga puntos y avisa al autor.

RESPECTO AL PROTOTIPO

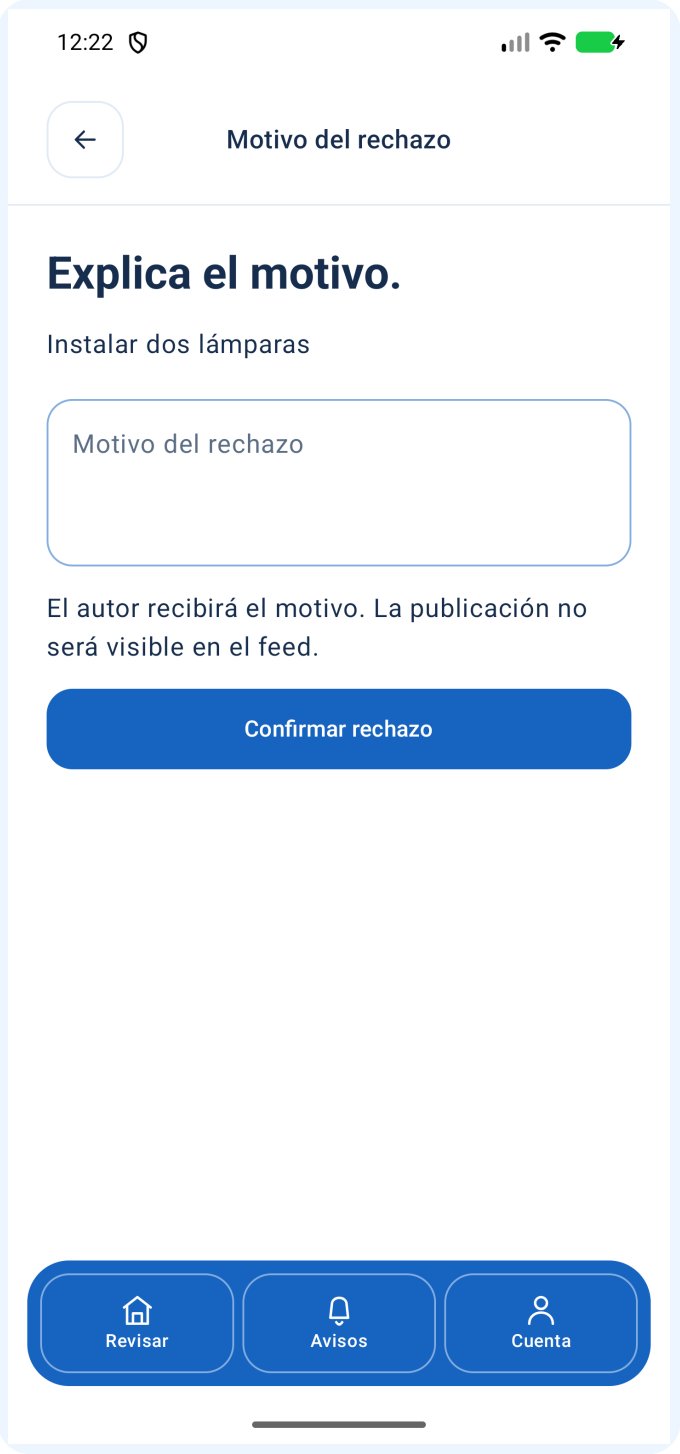
Se sustituyeron los botones informativos de la etapa anterior por operaciones del repositorio.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 41-mod-detail.png

03 · EVIDENCIAS FUNCIONALES

RF-37 · Rechazar con motivo obligatorio



IMPLEMENTACIÓN

Se exige un motivo de 5 a 500 caracteres. Se cambia a rechazada, se excluye del feed y se notifica al autor.

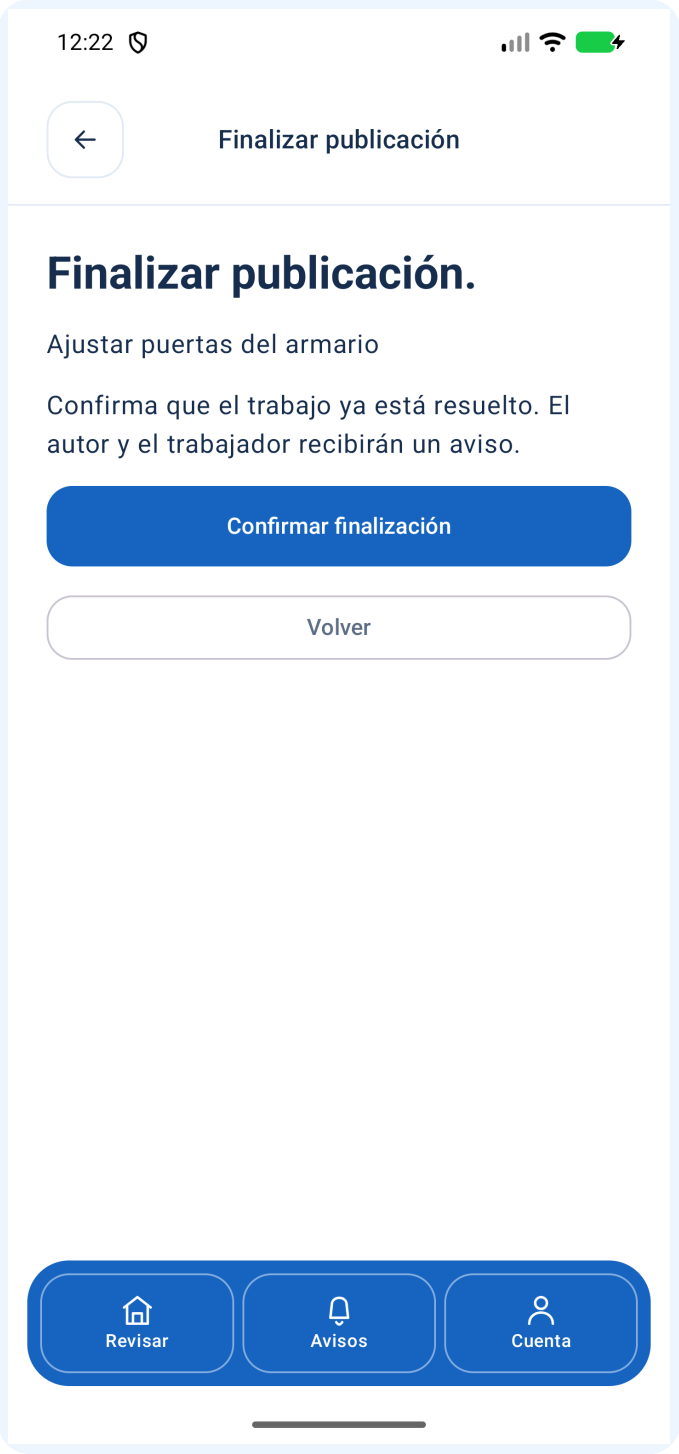
RESPECTO AL PROTOTIPO

La validación se ejecuta en el repositorio, incluso si se intenta invocar la acción sin completar el formulario.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 42-reject.png

RF-38 · Finalizar como moderador



IMPLEMENTACIÓN

El moderador puede finalizar cualquier publicación aún no finalizada, con una confirmación previa.

RESPECTO AL PROTOTIPO

La comprobación de rol se conserva en la operación de datos, además del acceso de la interfaz.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 43-mod-finish.png

RF-39 · Estados vacíos y navegación



IMPLEMENTACIÓN

Una búsqueda sin coincidencias presenta un mensaje y permite restablecer filtros. La navegación incluye botones de volver y barra inferior.

RESPECTO AL PROTOTIPO

El contenido tiene desplazamiento vertical y se adapta al espacio disponible y al teclado.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 44-empty.png

RF-40 · Avisos derivados de moderación



IMPLEMENTACIÓN

El autor consulta los avisos generados después de verificar o rechazar solicitudes durante la evaluación.

RESPECTO AL PROTOTIPO

La captura corresponde a acciones reales de la secuencia de prueba, no a un listado aislado del prototipo.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 46-notification-moderation.png

RF-41 · Marcar interés en una publicación



IMPLEMENTACIÓN

El botón Me interesa registra o retira el voto del usuario y actualiza el contador. No permite votar publicaciones propias ni otorgar puntos repetidos al alternar el voto.

RESPECTO AL PROTOTIPO

La interacción utiliza el estado compartido en memoria. Las pruebas unitarias verifican la prevención de puntos duplicados.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 11-detail.png

Arquitectura y condiciones de fase

Kotlin, Compose y MVVM

MainActivity aloja CommunityRoute. CommunityViewModel gestiona el estado de navegación y las acciones. CommunityRepository define el contrato en dominio; InMemoryCommunityRepository lo implementa en datos. Hilt vincula la interfaz y entrega una instancia compartida. StateFlow actualiza las pantallas.

Reglas y consistencia

Las restricciones de rol, propiedad, estados, duplicados y validación se verifican al operar los datos. Las ofertas, publicaciones, comentarios, avisos y reseñas usan identificadores; no son listas independientes sin relación.

Persistencia, imágenes, ubicación y permisos

Exentos por la consigna: base de datos, almacenamiento externo, carga de imágenes, permisos, mapas y sesiones. Se incluyen imagen ilustrativa, coordenadas de referencia y cuentas de prueba en memoria. El mapa opcional no condiciona el recorrido principal.

GitHub y contribuciones

El enlace del repositorio figura en la portada. Cada integrante debe realizar y conservar sus contribuciones reales con su identidad; la asistencia de IA no sustituye la participación del grupo. No se fabrican commits a nombre de los integrantes.

Legibilidad y presentación

Interfaz en español, fondo blanco, acentos y bordes azules, botones redondeados, una tarjeta por bloque, icono de ojo para contraseñas y mensajes emergentes. Se conserva scroll para pantallas pequeñas y teclado.

05 · DIFERENCIAS

Cambios frente a la primera fase

Identidad visual

Se pasó de las primeras propuestas oscuras, naranja o verde a la variante blanca y azul elegida por el usuario. Se redondearon controles y la barra inferior para separar opciones con claridad.

Tarjetas y encabezados

Se eliminaron recuadros duplicados en perfiles y reseñas. La campana se reservó para perfil/cuenta; las pantallas de recuperación no muestran avisos ajenos al flujo.

Datos conectados

El prototipo mostraba ejemplos estáticos. La entrega usa un repositorio en memoria: comentarios generan avisos, ofertas asignan trabajadores, moderación cambia la visibilidad y reseñas actualizan perfiles.

Formularios y confirmaciones

Se añadieron validaciones de contraseña, correo, campos obligatorios y presupuesto. Eliminar publicaciones/cuentas, aceptar ofertas y finalizar trabajos requieren acciones explícitas.

Simplificaciones de alcance

Registro sin mapa obligatorio; ciudad de referencia para solicitudes; imagen de ejemplo en lugar de subida externa; currículum estructurado en lugar de archivo obligatorio; recuperación con enlace de prueba. Estas diferencias responden a las exenciones de la consigna.

Servicios no afirmados como completos

No se afirma que existan autenticación de Google, correo real, backend, permisos o persistencia de sesiones. El estado de error de conexión es una pantalla de referencia, no una prueba de un servicio remoto.

06 · USO DE IA

Herramientas y propósito

Herramienta utilizada

Codex, asistente de programación de OpenAI, utilizado en esta conversación como agente principal. No se atribuye trabajo a otros agentes ni a herramientas de IA que no constan en el historial.

Apoyo en el diseño

Interpretación de requisitos y referencias visuales; adaptación del diseño blanco y azul; separación de tarjetas, navegación y mensajes de validación.

Apoyo en implementación

Generación y ajuste de Kotlin/Compose, ViewModels, modelos, repositorios e inyección Hilt/KSP. Implementación de flujos en memoria para la fase académica y restricciones de roles/propiedad.

Apoyo en verificación

Creación de pruebas unitarias y una prueba instrumentada. Lectura de errores de compilación, corrección de incompatibilidades y generación de capturas reales desde Android. Formateo con ktfmt; Gradle y Android Emulator son herramientas de desarrollo, no agentes de IA.

Apoyo documental

Organización de requisitos y evidencias, redacción del registro de cambios y del uso de IA. Generación del PDF con ReportLab y revisión visual de páginas renderizadas.

Límite de responsabilidad

El asistente apoya, pero el grupo debe comprender, revisar y ajustar el código. Este documento no certifica que los tres integrantes hayan hecho una revisión manual completa; esa validación humana corresponde al grupo antes de entregar.

06 · USO DE IA

Prompts relevantes del trabajo

Diseño y requisitos

“Debes hacer los mockups cumpliendo con todos los requisitos [...] que se vean más amigables [...] y elegante”. Propósito: derivar pantallas y jerarquía visual de la especificación.

Interacción y validaciones

“La contraseña debe tener [...] mínimo [...] mayúscula y número”; “el mensaje solo debe salir [...] cuando se registre”; “esto tiene que ser un ojo [...] ver contraseña”. Propósito: precisar validación y controles del formulario.

Navegación funcional

“Se ve estático, debes hacer que sea funcional la app, no solo [...] vistas”. Propósito: conectar pantallas, acciones y estados.

Arquitectura de clase

“Revisa esto y aprende”, junto con la guía de ViewModels/repositorios, y “organiza el proyecto como debe quedar”. Propósito: separar presentación, dominio, datos e inyección.

Alcance de esta entrega

Se proporcionó la consigna de fase dos y se confirmó: “Sí, completa la app y el PDF”. Propósito: completar flujos con datos locales y producir evidencias por requisito.

Trazabilidad

Los fragmentos se normalizan en ortografía y se acortan para facilitar la lectura; no representan un registro exhaustivo de todos los mensajes. Los resultados se validan contra el comportamiento de la app y las pruebas, no contra la mera aceptación de una respuesta de IA.

Dificultades y correcciones

Pantallas sin operaciones reales

La versión inicial validaba formularios, pero no creaba cuentas ni ejecutaba moderación. Se implementó un repositorio compartido en memoria y se conectaron registro, acceso, revisión, ofertas, comentarios y reseñas.

Mapa en blanco

Se detectó un área de mapa sin contenido. Se ajustaron el tamaño del WebView, los recursos Leaflet incluidos y la actualización del tamaño. Se verificó en el emulador que tocar otro lugar cambiaba las coordenadas.

Errores durante la reorganización

La extracción de clases produjo imports duplicados y desajustes en dobles de prueba. Se corrigieron imports, constructores e interfaces; se repitió la compilación y las pruebas.

Compatibilidad de pruebas Android

La infraestructura inicial de pruebas usaba versiones antiguas y falló en Android 17 al acceder a InputManager. Se actualizaron las bibliotecas de pruebas y se volvió a ejecutar el recorrido instrumentado.

Compilación de estados opcionales

Al conectar el flujo completo se detectaron accesos a publicaciones que podían ser nulas. Se añadieron comprobaciones explícitas antes de consultar sus datos.

Prevención de resultados engañosos

Se diferenciaron el enlace de recuperación simulado, las imágenes de ejemplo y los datos en memoria de servicios remotos reales. Las capturas se tomaron de Android, no del prototipo HTML.

07 · VERIFICACIÓN

Cómo se comprobó el resultado

Compilación

Se ejecutó Gradle para generar la aplicación debug y el paquete de pruebas Android. Resultado final: compilación satisfactoria.

Pruebas unitarias

43 pruebas aprobadas, 0 fallos. Incluyen validaciones previas y pruebas nuevas de registro, recuperación, rol, moderación, notificaciones, votos, ofertas, reseñas, eliminación y estado pendiente.

Prueba instrumentada

1 recorrido(s) instrumentado(s) aprobado(s), 0 fallos finales. Se operan los formularios de registro, login y reseña mediante controles Compose; el resto del recorrido se dirige mediante el mismo ViewModel utilizado por la interfaz. No se presenta como prueba manual exhaustiva de cada control.

Capturas y trazabilidad

Las imágenes se obtuvieron con la captura del emulador después de renderizar cada pantalla. Se verifican estados de registro, asignación, finalización, reseña y moderación durante la secuencia. Las fichas RF-01 a RF-41 relacionan requisitos y capturas.

Reproducción

Comandos: ./gradlew :app:assembleDebug :app:testDebugUnitTest
Prueba de dispositivo: :app:connectedDebugAndroidTest con la clase DeliveryEvidenceTest. Consultar docs/FASE_2.md en el repositorio.

Revisión humana pendiente del grupo

Antes de presentar: ejecutar el recorrido sugerido; explicar modelos, repositorio, ViewModel y roles; revisar las diferencias; confirmar el enlace/versionado en GitHub; verificar que las contribuciones de cada integrante estén registradas. No sustituir esa revisión por las pruebas automatizadas.

08 · REFERENCIAS

Fuentes y archivos de apoyo

Repositorio

github.com/JDFaFa/ServiNow

Guía de clase

Carlos Andrés Florez V. Arquitectura MVVM y Repositorios en Android. Universidad del Quindío.
Guía de ViewModels y repositorios

Documentación de herramientas

Hilt: configuración Gradle
Android Developers: pruebas de Compose

Archivos para estudiar

MainActivity.kt; features/community/CommunityScreen.kt; features/community/CommunityViewModel.kt; domain/model/Community.kt; domain/repository/CommunityRepository.kt; data/repository/InMemoryCommunityRepository.kt; di/RepositoryModule.kt.

Pruebas y documentación

CommunityTest.kt; RepositoryBoundaryTest.kt; DeliveryEvidenceTest.kt; docs/FASE_2.md; docs/ARQUITECTURA.md. Los nombres de carpetas reflejan la separación de responsabilidades; el código fuente permite revisar la implementación completa.